ClaimGuardian
AI-Assisted Pre-Submission Validation System for Health Insurance Claims in India

Synopsis Report submitted in partial fulfillment of the requirement for the degree of

Bachelor of Technology in Information Technology

Submitted By

Sarthak Kulkarni
Shreyas Khobragade
Rohit Gupta

Under the Guidance of

Dr. Vidya Chitre
Department of Information Technology

Vidyalankar Institute of Technology
Wadala(E), Mumbai 400 037

University of Mumbai
2025–26

CERTIFICATE OF APPROVAL

For Project Synopsis

This is to certify that

Sarthak Kulkarni

Shreyas Khobragade

Rohit Gupta

have successfully carried out Project Synopsis work entitled

ClaimGuardian

AI-Assisted Pre-Submission Validation System for Health Insurance Claims in India

in partial fulfillment of degree course in Information Technology
as laid down by University of Mumbai during the academic year 2025–26

Under the Guidance of
Dr. Vidya Chitre

Signature of Guide	Head of Department
(Dr. Vidya Chitre)	(Dr. Vidya Chitre)
Examiner 1	Examiner 2

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea, data, fact, or source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

S. No.	Name of Student	Roll No.	Signature
1	Sarthak Kulkarni	23101B0019	
2	Shreyas Khobragade	23101A0033	
3	Rohit Gupta	23101A0019	

Date: 26-04-2026

Acknowledgements

We express our profound gratitude to all those who have supported, guided, and inspired us throughout the conception and development of this capstone project.

Our sincere thanks to Dr. Sangeeta Joshi, Principal of Vidyalankar Institute of Technology, for her visionary leadership, unwavering commitment to academic excellence, and for fostering an environment that encourages innovation and hands-on learning. We are deeply grateful to Vidyalankar Institute of Technology for providing the academic infrastructure, computational resources, and intellectual ecosystem essential to the successful execution of this project.

We extend our heartfelt appreciation to our project guide and Head of the Department, Dr. Vidya Chitre, for her steadfast mentorship, technical insights, and continuous encouragement at every stage of this work. Her constructive feedback, patience, and scholarly guidance were instrumental in shaping the architecture, methodology, and overall quality of this project. We also acknowledge the faculty members and staff of the Department of Information Technology for their academic support, valuable suggestions, and for cultivating a rigorous learning environment that continually challenges us to excel.

We are deeply thankful to Mpathy Health Solutions Private Limited for sharing their domain expertise, providing practical insights into health-insurance claim adjudication workflows, and supplying anonymized sample materials. Their industry perspective was invaluable in grounding this project in real-world operational challenges and ensuring its technical and regulatory relevance.

We gratefully acknowledge our families and friends for their unwavering emotional support, patience, and encouragement throughout the demanding phases of development, testing, and documentation. Their belief in our capabilities kept us motivated during intensive work cycles and complex problem-solving stages.

We also thank our fellow students, peers, and academic collaborators for their collaborative spirit, constructive discussions, and shared learning experiences that enriched our understanding of applied AI and enterprise software engineering. Finally, we extend our sincere appreciation to the readers, evaluators, and academic community for their time, engagement, and commitment to advancing responsible, explainable AI in healthcare and insurance technology.

This project stands as a collective achievement, and we are honored to have been guided and supported by such an inspiring network of mentors, industry professionals, educators, and loved ones.

Table of Contents

Declaration i

Acknowledgements ii

List of Tables vii

List of Figures ix

Organization Profile x

Abstract xi

1 Introduction 1

2 Aim and Objectives 3

3 Literature Surveyed 4

4 Problem Statement and Motivation 6

4.1 Problem Statement 6

4.2 Need and Motivation 7

4.2.1 User-Centric Need: Reducing Avoidable Claim Friction 7

4.2.2 Operational Need: Improving Intake Efficiency for Insurers and TPAs . 7

4.2.3 Regulatory Need: Aligning with DPDP Act 2023 and IRDAI Guidance 8

4.2.4 Academic and Technical Need: Advancing Document Intelligence for Indian Contexts 8

4.2.5 Strategic Need: Building a Scalable Foundation for Enterprise Adoption 9

4.3 Evidence Base and Research Anchors 9

5 Scope 10

5.1 In-Scope: Core Deliverables (Capstone MVP) 10

5.1.1 AI-Native Document Intelligence Pipeline 10

5.1.2 Intelligent Validation and Reasoning 11

5.1.3 User Experience and Output 11

5.1.4 Privacy, Architecture and MLOps 12

5.2 Out-of-Scope: Explicitly Excluded Capabilities 12
5.3 Deferred for Enterprise Readiness (Acknowledged but Not Implemented) . . . 14
5.4 Scope Boundaries: Regulatory and Ethical Guardrails 15
5.5 Feasibility Alignment: 8-Month Student Team Constraints 16

6 Proposed System **17**
6.1 System Overview and Architectural Vision 17
6.2 End-to-End Operational Workflow 18
6.3 AI-Native Pipeline Architecture 18
6.4 Privacy, Security and Compliance by Design 20
6.5 Technical Stack and Deployment Strategy 20
6.6 Key Differentiators and Alignment with Academic and Industry Goals 21
6.6.1 Scope Reinforcement for Academic Integrity 22

7 Methodology (Approach to Solve the Problem) **23**
7.1 Domain-Driven Problem Framing and Scope Calibration 23
7.2 Privacy-by-Design Data Curation and PHI-Safe Processing 23
7.3 Hybrid AI Pipeline Construction (Vision to Rules to Reasoning) 24
7.4 Explainable Output Generation and Human-in-the-Loop Validation 24
7.5 MLOps-Driven Experimentation and Reproducible Benchmarking 25
7.6 Incremental Integration, Adversarial Testing and Privacy Hardening 25
7.7 Deployment, Evaluation and Knowledge Transfer 25

8 Analysis **27**
8.1 Process Model Used for the Project 27
8.1.1 Rationale for Process Model Selection 27
8.1.2 Process Workflow Diagram 28
8.1.3 Key Process Artifacts 28
8.2 Feasibility Study 29
8.2.1 Technical Feasibility 29
8.2.2 Operational Feasibility 30
8.2.3 Economic Feasibility (Zero-Budget Strategy) 30
8.2.4 Schedule Feasibility 31

8.2.5 Legal and Regulatory Feasibility 32
8.3 Cost Analysis 32
8.3.1 Resource Breakdown (Zero-Budget Execution) 32
8.3.2 Human Effort Estimation (Student Team) 33
8.3.3 Cost-Benefit Summary 33
8.4 Timeline Chart 34
8.4.1 High-Level Gantt Chart (26-Week Capstone Timeline) 34
8.4.2 Module-Level Milestone Map 35
8.4.3 Critical Path and Risk Mitigation 36

9 System Design **37**
9.1 Data Flow Diagram (DFD) 37
9.1.1 Context Diagram (Level 0) 37
9.1.2 Level 1 DFD: Core Processing Pipeline 38
9.1.3 Level 2 DFD: Privacy-Preserving Data Handling 40
9.2 UML Diagrams 41
9.2.1 Use Case Diagram 41
9.2.2 Class Diagram (Core Domain Model) 41
9.2.3 Sequence Diagram: End-to-End Validation Workflow 42
9.3 Enhanced Entity-Relationship (EER) Diagram 43
9.3.1 EER Design Rationale 43
9.3.2 Database Implementation Notes 44
9.4 Design Validation Against Requirements 44

10 Hardware and Software Requirements (Technology Stack) **46**
10.1 Hardware Requirements 46
10.1.1 Development Environment (Student Team) 46
10.1.2 Testing & Evaluation Environment 46
10.1.3 Deployment Environment (Capstone Demo) 47
10.2 Software Requirements 47
10.2.1 AI / ML Stack (Core Intelligence Layer) 47
10.2.2 Backend & API Layer 48
10.2.3 Frontend & User Interface 49

10.2.4 Infrastructure & DevOps 50
10.2.5 Privacy, Security & Compliance 50
10.3 Zero-Budget Execution Strategy 51
10.4 Upgrade Path for Enterprise Deployment 52
10.5 Compatibility & Dependency Management 53

References **54**

List of Tables

4.1 Common avoidable discrepancies in health-insurance claim preparation 7
4.2 Evidence base used to motivate the proposed system 9
5.1 In-scope document intelligence capabilities 10
5.2 In-scope validation and reasoning capabilities 11
5.3 In-scope user experience and output capabilities 11
5.4 In-scope privacy, architecture, and MLOps capabilities 12
5.5 Explicitly excluded capabilities 12
5.6 Capabilities deferred for enterprise readiness 14
5.7 Regulatory and ethical scope boundaries 15
5.8 Feasibility alignment with capstone constraints 16
6.1 Architectural layers of the proposed ClaimGuardian system 17
6.2 Privacy, security, and compliance controls in the proposed system 20
6.3 Technical stack proposed for ClaimGuardian 21
6.4 Key differentiators of the proposed ClaimGuardian system 22
8.1 Process model adaptations for ClaimGuardian 27
8.2 Key process artifacts for delivery, privacy, and evaluation 29
8.3 Technical feasibility assessment 29
8.4 Operational feasibility assessment 30
8.5 Zero-budget economic feasibility assessment 31
8.6 Schedule feasibility assessment 31
8.7 Legal and regulatory feasibility assessment 32
8.8 Zero-budget resource breakdown 32
8.9 Human effort estimation for the student team 33
8.10 Cost-benefit summary 33
8.11 Module-level milestone map 35
8.12 Critical path risks and mitigation strategy 36
9.1 External entities and data flows in the context diagram 37
9.2 EER design rationale 43
9.3 Design validation against requirements 44
10.1 Development environment requirements for the student team 46

10.2 Testing and evaluation environment 47
10.3 Deployment environment for the capstone demo 47
10.4 AI / ML stack for the core intelligence layer 47
10.5 Backend and API layer requirements 49
10.6 Frontend and user-interface requirements 49
10.7 Infrastructure and DevOps requirements 50
10.8 Privacy, security, and compliance requirements 50
10.9 Zero-budget execution strategy 51
10.10 Upgrade path for enterprise deployment 52
10.11 Compatibility and dependency management 53

List of Figures

6.1 Simplified AI-native pipeline flow for ClaimGuardian 19

8.1 Process workflow diagram for the ClaimGuardian delivery plan 28

8.2 High-level Gantt chart for the 26-week ClaimGuardian capstone timeline 34

9.1 Context diagram for ClaimGuardian (Level 0 DFD) 37

9.2 Core processing pipeline (Level 1 DFD) 39

9.3 Privacy-preserving data handling (Level 2 DFD) 40

9.4 Use case view of ClaimGuardian actors and interactions 41

9.5 Core domain model for sessions, claims, rules, and reports 41

9.6 End-to-end sequence for upload, validation, explanation, and export 42

9.7 Enhanced entity-relationship model for ClaimGuardian 43

Organization Profile

Mpathy Health Solutions Private Limited is a relatively new organization incorporated on 3 December 2025. Based in Pune, Maharashtra, the company operates in the domain of management consultancy within the health-insurance sector. Its work focuses on simplifying the technical and procedural complexity that often causes confusion during the health-insurance claim journey.

The organization was founded to address the practical problems created by complex jargon, unclear policy interpretation, rejected claims, unreasonable deductions, and settlement delays. Its services include end-to-end claim consultation for disputed or delayed claims, guidance on policy terms and conditions, and assistance in understanding technical aspects such as sum insured and related benefit clauses. This orientation aligns closely with the motivation behind the proposed ClaimGuardian system, which also aims to reduce avoidable claim-submission errors through clearer validation and explanation.

Another important aspect of the organization is its training and mentorship role in the insurance ecosystem. It conducts specialized learning programs for insurance agents and sales forces and has reportedly trained more than 50,000 agents. The core team includes doctors with over a decade of experience specifically in health-insurance claim departments, which strengthens the organization's practical understanding of documentation issues, claim workflows, and user pain points.

The registered office of the company is Office No. 4, Second Floor, Rokade Heights, Paud Road, Kothrud, Pune, Maharashtra 411038. The board of directors includes Sachin Shrikant Mahajan and Saurabh Mehta. The company is listed as an unlisted private company with active e-filing status. For this academic project, the organization serves as the external domain partner by contributing contextual understanding of claims processing and helping keep the project grounded in real-world healthcare-insurance challenges.

Abstract

Health insurance claims are frequently delayed or rejected at the first submission stage because of avoidable issues such as missing information, incorrect dates, incomplete documents, and misunderstanding of policy limits. In the present ecosystem, the validation logic used by insurers and Third-Party Administrators is usually not visible to policyholders, which results in repeated resubmissions and higher manual effort. This project proposes **ClaimGuardian**, an AI-assisted pre-submission validation system that analyzes claim-related documents before formal submission and highlights potential risks in simple language.

The proposed system accepts documents such as policy papers, hospital bills, investigation reports, and discharge summaries; extracts important fields using document intelligence; masks sensitive personal information; applies deterministic validation rules; and generates a user-friendly report with warnings, explanations, and corrective suggestions. The solution is envisioned as a privacy-first and explainable platform in which every major flag can be traced either to an extracted document field or to a clearly stated rule condition. Such an approach is important in the insurance domain because users are more likely to trust automated feedback when the basis of the warning is transparent and actionable.

In addition to document understanding, the project focuses on reducing the communication gap between technical validation logic and non-technical users. Instead of returning only a binary pass-or-fail outcome, ClaimGuardian aims to provide graded risk indicators, plain-language summaries, and next-step guidance such as which document may be missing, which field appears inconsistent, or which policy condition requires closer review. This makes the system more useful as a claim-preparation assistant rather than merely a backend screening tool.

The expected outcome is a working prototype that improves first-time claim quality, reduces avoidable submission mistakes, supports users in preparing cleaner claim packets, and demonstrates the practical use of artificial intelligence in insurance operations. From an academic perspective, the project also showcases the integration of OCR, multimodal extraction, de-identification, explainable rule validation, and report generation in one coherent application relevant to healthcare and financial services.

1 Introduction

Health insurance in India has experienced rapid expansion, with coverage extending to over 450 million individuals across public and private schemes. Despite this growth, the claim submission process remains a critical operational and experiential bottleneck. Policyholders frequently encounter first-pass rejections or processing delays not because their claims are fundamentally invalid, but because submitted documentation contains preventable gaps: missing identifiers, inconsistent admission or discharge dates, room-rent cap violations, incomplete policy schedules, or misaligned coverage clauses. Industry analyses and IRDAI settlement reports consistently indicate that a substantial share of intake-level rejections stem from these structural and administrative discrepancies rather than clinical or eligibility disputes.

Compounding the issue is the opacity of claim validation logic. Insurers and Third-Party Administrators (TPAs) evaluate claims through internal, rule-based adjudication pipelines that are not visible to users at the point of submission. As a result, policyholders, caregivers, or hospital billing staff submit claims blindly, discover errors only after formal review, and enter costly resubmission loops. This lack of transparency increases manual rework for operations teams, delays settlements, and erodes consumer confidence in insurance workflows.

Concurrently, recent advances in artificial intelligence have matured to a point where document intelligence, privacy-preserving natural language processing, and explainable automation can be integrated into practical, auditable systems. Modern Vision-Language Models (VLMs) demonstrate strong layout-aware understanding of scanned, multilingual, and table-heavy documents. Transformer-based Named Entity Recognition (NER) and Retrieval-Augmented Generation (RAG) enable systems to extract structured fields, cross-reference policy clauses, and generate grounded, human-readable explanations. When combined with deterministic validation engines and ephemeral, privacy-by-design architectures, these technologies make it feasible to build a transparent pre-check layer that mirrors insurer validation logic without compromising data security or regulatory compliance.

ClaimGuardian is proposed as an AI-augmented, pre-submission validation system designed specifically for the Indian health-insurance context. The platform ingests scanned policy documents, hospital bills, and discharge summaries; extracts structured claim fields using multimodal document understanding; applies policy-grounded validation rules; and returns actionable risk indicators alongside plain-language, policy-cited explanations. Crucially, the system operates

strictly as a pre-check assistant and explicitly does not perform claim adjudication, fraud detection, or clinical judgment. Privacy is engineered into the architecture through ephemeral session processing, PHI anonymization (for example, Microsoft Presidio with custom recognizers), and strict adherence to data-minimization principles aligned with India's Digital Personal Data Protection (DPDP) Act, 2023.

From an academic standpoint, this project integrates multimodal document intelligence, explainable AI reasoning, privacy engineering, and MLOps-ready architecture into a single, evaluable pipeline. From an industry perspective, it addresses a documented intake bottleneck by improving first-pass claim quality, reducing manual rework, and aligning with emerging regulatory expectations for transparency and consumer empowerment. The system is engineered with modular, enterprise-grade boundaries while remaining scoped for rigorous capstone delivery: each AI component is independently testable, fallback chains are explicitly defined, and all outputs are confidence-weighted and human-verifiable.

By transforming raw, heterogeneous claim documents into structured signals and policy-grounded recommendations, ClaimGuardian aims to bridge the transparency gap between policyholders and insurers. The project demonstrates how responsible, privacy-first AI can be applied to high-stakes, real-world workflows without overpromising automation or compromising regulatory trust.

2 Aim and Objectives

Aim

To design, develop, and evaluate an AI-augmented, privacy-first pre-submission validation platform for Indian health-insurance claims that improves first-pass submission quality, surfaces policy-aligned risks prior to formal adjudication, and delivers transparent, explainable guidance to users while adhering to data-minimization and India's DPDP Act 2023 compliance principles.

Objectives

- To ingest and automatically classify heterogeneous claim documents such as policy schedules, hospital bills, and discharge summaries using machine-learning-based document analysis, thereby establishing structured routing for downstream AI processing without relying on manual user labeling.
- To extract structured claim fields from scanned, noisy, or semi-structured documents through a hybrid AI pipeline involving Vision-Language Models, OCR ensembles, and transformer-based Named Entity Recognition, with per-field confidence scoring and graceful-degradation fallbacks to ensure reliability across variable document quality.
- To detect, mask, and verify removal of Personal Health Information (PHI) and Indian Personally Identifiable Information, such as Aadhaar, PAN, and policy identifiers, using Microsoft Presidio and domain-tuned recognizers so that all sensitive data is anonymized prior to storage, logging, or downstream inference in alignment with DPDP Act 2023 data-minimization mandates.
- To validate extracted claim data against deterministic, policy-grounded business rules such as waiting periods, room-rent caps, document completeness, and co-pay limits, ensuring that all checks remain auditable, version-controlled, and explicitly traceable to source policy clauses rather than opaque model predictions.
- To generate plain-language, policy-cited explanations and corrective guidance for flagged discrepancies using retrieval-augmented generation (RAG) and constrained LLM prompting, with explicit grounding requirements and deterministic output settings to prevent hallucination and maintain regulatory trust.
- To deliver an interactive review workflow that enables field verification, confidence-weighted correction, and export of a structured, timestamped validation report, culminating in a privacy-hardened ephemeral session architecture with metadata-only audit logging suitable for academic evaluation and future enterprise extensibility.

3 Literature Surveyed

The uploaded literature review and supporting project documents indicate that a viable pre-submission claim-validation system cannot depend on a single end-to-end model. Instead, it must combine document intelligence, structured field extraction, policy-grounded validation, privacy engineering, and explainable reporting in one coordinated pipeline [1, 2, 3, 4, 5, 6, 7]. The survey also makes it clear that Indian health-insurance claims form a particularly difficult AI problem because the inputs are noisy, multilingual, and layout-heavy, while the outputs must remain auditable and safe for high-stakes use.

1. **Document intelligence for noisy claim packets:** Literature on OCR and multimodal extraction shows that hospital bills, discharge summaries, and policy schedules frequently contain tables, stamps, mixed print quality, and inconsistent templates. The uploaded review therefore recommends a hybrid extraction layer that combines OCR with layout-aware vision-language models so that both text and document structure can be captured more reliably than with plain OCR alone [5, 6, 8, 1].

2. **Structured field extraction and schema grounding:** Prior work repeatedly emphasizes that raw OCR output is not directly useful for validation unless it is mapped into a stable schema. Frequently required entities include member identifiers, admission and discharge dates, billed amounts, room-rent details, diagnosis information, and policy clauses. This supports a pipeline in which extraction is followed by normalization, confidence scoring, and user correction before any rule engine is applied [5, 6, 2, 3].

3. **Hybrid reasoning instead of opaque prediction:** The strongest architectural conclusion from the literature is that insurance validation should not rely on a black-box model alone. Straightforward checks such as waiting periods, document completeness, date consistency, and room-rent caps are better handled by deterministic rules, while ambiguous narrative text can be interpreted through a grounded LLM or retrieval-based component. This neuro-symbolic split improves auditability and reduces hallucination risk [9, 1, 2, 4].

4. **Privacy-preserving processing as a first-class requirement:** Claim documents contain PHI and Indian identifiers such as Aadhaar, PAN, and policy numbers, making de-identification essential before storage or downstream inference. Microsoft Presidio is identified as a practical starting point, but the literature also points to the need for domain-specific recognizers and ephemeral-session processing so that privacy is built into the architecture

rather than added later [7, 10, 1, 3].

5. **Explainability, human oversight, and evaluation:** A recurring theme is that detection accuracy alone is insufficient in a claim-support system. Users and reviewers must be able to see why a field was flagged, which rule or policy condition was triggered, and what corrective action is suggested. The surveyed material also points toward human-in-the-loop review, grounded explanations, and evaluation beyond raw accuracy, including extraction quality, rule precision, privacy leakage, and end-to-end usefulness [9, 8, 1, 3].

From this survey, the project direction chosen for ClaimGuardian is a hybrid AI architecture: multimodal extraction for difficult documents, schema-based normalization, deterministic validation for auditable checks, and constrained explanation generation for user-facing guidance. This combination is better aligned with Indian health-insurance workflows than either a pure OCR pipeline or a fully autonomous black-box decision model [1, 9, 7, 4, 3].

4 Problem Statement and Motivation

4.1 Problem Statement

In India's health-insurance ecosystem, policyholders still lack pre-submission validation tools capable of identifying common and avoidable claim errors before a claim is formally submitted to insurers or Third-Party Administrators (TPAs). Reporting based on FY2023–24 IRDAI data indicates that roughly 11% of health-insurance claims were rejected and about 6% remained pending at the end of the reporting period, implying that only around 83% reached settlement within the reporting cycle [11, 12]. A meaningful share of this friction arises not from fundamental ineligibility, but from structural and administrative discrepancies that could be detected and corrected earlier in the workflow [1, 3].

Critically, claim-validation logic remains opaque to users at the point of submission. Insurers and TPAs rely on internal rule-based adjudication practices, but these checks are not visible to policyholders, caregivers, or hospital billing staff who prepare claim packets. This creates a submit-and-wait cycle in which errors are discovered only after formal review, leading to repeated resubmissions, longer turnaround times, and additional manual effort for both users and intake teams [1, 13].

The regulatory environment makes the challenge more demanding. The Digital Personal Data Protection Act, 2023 frames digital personal-data processing around lawful purpose, notice, consent, and data-fiduciary obligations, while IRDAI requires denial or repudiation communications to specifically state the reasons and corresponding policy conditions [7, 14]. At the same time, generic OCR pipelines struggle with noisy, multilingual, and layout-heavy Indian medical documents, and cloud-only AI workflows raise privacy concerns in a regulated environment [5, 6, 8, 1, 10].

Central Design Question: How can a privacy-first, explainable pre-submission validation system help Indian health-insurance policyholders identify and correct common claim errors before formal submission, while remaining aligned with DPDP Act 2023 obligations and IRDAI transparency expectations?

Table 4.1: Common avoidable discrepancies in health-insurance claim preparation

Error Category	Example	Impact
Incomplete Documentation	Missing discharge summary, unsigned forms, absent policy schedule pages	Claim returned for resubmission or delayed during intake review.
Date Inconsistencies	Admission date after discharge date; policy start date after treatment date	Eligibility disputes, manual escalation, and processing delays.
Policy-Limit Violations	Room rent exceeding policy cap; procedure not clearly covered under the selected plan	Partial approval, deduction, or need for clarification.
Formatting and OCR Failures	Poor-quality scans, handwritten entries, multilingual text, broken page alignment	Extraction errors and additional manual rework.
Terminology Mismatches	Diagnosis wording or treatment description not aligned with policy language	Ambiguity in review and inconsistent interpretation.

4.2 Need and Motivation

4.2.1 User-Centric Need: Reducing Avoidable Claim Friction

A recent claimant survey reported that 43% of policyholders who had filed health-insurance claims in the preceding three years experienced processing difficulties, including delays, partial approvals, or repeated requests for additional documentation [13]. During already stressful medical episodes, repeated resubmissions create financial, emotional, and temporal burdens for patients and families. A pre-check layer can shift the workflow from reactive correction to proactive preparation by giving users actionable, policy-linked feedback before submission.

4.2.2 Operational Need: Improving Intake Efficiency for Insurers and TPAs

FY2023–24 reporting also indicates that the value of disallowed and repudiated health-policy claims increased by 19.10% year over year, signalling a substantial rework burden for insurer intake and claims-processing teams [15, 11]. Avoidable submission errors consume review capacity that could otherwise be focused on complex or high-value adjudication tasks. Cleaner

first-pass submissions therefore support both operational efficiency and better customer experience.

4.2.3 Regulatory Need: Aligning with DPDP Act 2023 and IRDAI Guidance

Any AI-assisted claims tool in India must be engineered for responsible data handling and transparent communication. A pre-submission validator that uses ephemeral sessions, masks PHI, and limits downstream exposure of personal data is well aligned with the privacy-by-design direction encouraged by the DPDP framework [7, 10, 1]. Similarly, an explainable validation layer that links warnings to specific rule checks or policy conditions supports the IRDAI expectation that claim decisions be communicated with explicit reasons and policy references [14].

4.2.4 Academic and Technical Need: Advancing Document Intelligence for Indian Contexts

Much of the document-intelligence literature is built around cleaner, more standardized, Latin-script documents, whereas Indian medical and insurance records are frequently noisy, multilingual, table-heavy, and visually inconsistent [1, 5]. Recent vision-language models offer strong layout-aware extraction capabilities, but their robustness under real-world perturbations remains an open research concern [6, 8]. This project contributes academically by:

- evaluating hybrid OCR plus VLM extraction strategies on Indian claim documents,
- designing a neuro-symbolic validation pipeline that balances deterministic rules with LLM flexibility,
- implementing privacy-preserving NLP workflows suited to regulated insurance data, and
- defining reproducible evaluation metrics for extraction accuracy, explanation grounding, and end-to-end validation usefulness.

The use of retrieval-augmented generation is especially relevant because prior work shows that external retrieval can improve grounding and reduce unsupported generation in knowledge-intensive NLP tasks [9].

4.2.5 Strategic Need: Building a Scalable Foundation for Enterprise Adoption

Although the work is scoped as a capstone project, the target architecture is intentionally modular and enterprise-conscious: ephemeral sessions, metadata-only audit trails, containerized deployment, and a clear separation between extraction, validation, and explanation layers [4, 3, 1]. By restricting the system to pre-submission validation rather than claim adjudication, the project remains feasible within the academic timeline while still demonstrating a realistic pathway for later industry deployment.

4.3 Evidence Base and Research Anchors

The problem framing and proposed architecture are not based on intuition alone; they are grounded in a combination of regulatory sources, industry reports, and recent AI research on document understanding, retrieval grounding, and explainable validation. Table 4.2 consolidates the most relevant evidence anchors that justify why ClaimGuardian focuses on pre-submission claim quality, privacy-preserving processing, and policy-grounded assistance.

Table 4.2: Evidence base used to motivate the proposed system

Claim	**Supporting Evidence**
Roughly 11% of health-insurance claims were rejected and about 6% remained pending in FY2023–24	IRDAI Annual Report 2023–24 and India Today summary based on IRDAI data [11, 12].
43% of recent claimants reported claim-processing difficulties	Economic Times survey-based report on claimants' experience [13].
IRDAI requires repudiation communication to state reasons and corresponding policy conditions	Official IRDAI FAQ on health-insurance regulations [14].
DPDP Act 2023 frames digital personal-data processing around lawful and accountable handling	Official text of the Digital Personal Data Protection Act, 2023 [7].
Indian medical claim documents are noisy, multilingual, and structurally difficult to parse	Uploaded framework review together with research on form understanding and layout-aware extraction [1, 5, 6].
VLM robustness and explanation grounding remain important technical concerns	VLM-RobustBench and Retrieval-Augmented Generation literature [8, 9].

5 Scope

This section defines the boundaries of the ClaimGuardian project by specifying what will be delivered within the eight-month capstone timeline, what is explicitly excluded to preserve focus, and which capabilities are acknowledged as important for enterprise deployment but intentionally deferred for future work. The scope is deliberately constrained so that the project remains academically feasible while still demonstrating technical rigor and practical relevance to Indian health-insurance workflows [4, 3, 1].

5.1 In-Scope: Core Deliverables (Capstone MVP)

The following capabilities are within scope and together define the minimum acceptable capstone deliverable for academic submission and industry demonstration. These in-scope modules reflect the hybrid design justified in the literature survey: AI-assisted document understanding, deterministic validation, grounded explanation, and privacy-by-design processing [1, 6, 10, 9].

5.1.1 AI-Native Document Intelligence Pipeline

Table 5.1: In-scope document intelligence capabilities

Capability	**Description**	**Success Criteria**
Document Classification	Auto-classify uploaded documents such as hospital bills, discharge summaries, and policy documents using a fine-tuned DistilBERT or LayoutLM family model.	$\geq$ 90% F1-score on a held-out test set of Indian medical documents.
Vision-Language Extraction	Extract structured claim fields through local vision-language inference with a layout-aware model such as Qwen2.5-VL.	$\geq$ 85% token-level F1 on field extraction, along with per-field confidence scores.
Fallback Extraction Chain	Provide graceful degradation from VLM extraction to NER, OCR, and finally manual entry whenever confidence is low.	System remains functional even if the primary VLM fails, and users can correct extracted values before validation.
PHI Detection and Anonymization	Run Microsoft Presidio with custom Indian PII recognizers for identifiers such as Aadhaar, PAN, and IFSC before downstream processing.	0% PHI leakage in logs and generated outputs when tested against an adversarial PHI corpus.

5.1.2 Intelligent Validation and Reasoning

Table 5.2: In-scope validation and reasoning capabilities

Capability	Description	Success Criteria
Policy Rule Mining	Use a local LLM from the LLaMA or Mistral class to extract validation rules from uploaded policy documents and attach source citations.	$\geq$ 85% precision on human-evaluated rule extraction, with accepted rules stored as executable Python logic.
Deterministic Rule Engine	Apply extracted rules to claim data for checks such as waiting periods, room-rent caps, document completeness, and co-pay limits.	100% rule traceability to policy text and at least $\geq$ 85% precision on validation accuracy.
ML-Based Risk Prediction	Use a Gradient Boosting classifier to estimate claim-rejection risk from extracted features and rule violations, with post-hoc interpretability.	$\geq$ 80% ROC-AUC on a held-out test set and SHAP-based feature attribution for explanations.
RAG-Grounded Explanations	Generate plain-language explanations by retrieving relevant policy clauses through semantic search and grounding the LLM response in those clauses.	$\geq$ 80% grounding score against gold explanations and 100% traceability of claim feedback to source policy text.

5.1.3 User Experience and Output

Table 5.3: In-scope user experience and output capabilities

Capability	Description	Success Criteria
Interactive Review Interface	Present an editable review form showing extracted fields together with confidence indicators such as high-confidence, warning, or low-confidence states.	Users can successfully correct at least $\geq$ 90% of low-confidence fields, and client-side validation blocks invalid submissions.
Results Dashboard	Display validation output through a risk gauge, rule flags, policy-grounded explanations, and corrective recommendations.	Dashboard renders all rule outcomes correctly, and explanation readability is rated at least 4.0/5.0 in user evaluation.
PDF Report Export	Provide a downloadable validation report containing extracted fields, flags, explanations, timestamp, and session identifier.	Report is human-readable, contains complete validation results, and includes no PHI.

5.1.4 Privacy, Architecture and MLOps

Table 5.4: In-scope privacy, architecture, and MLOps capabilities

Capability	Description	Success Criteria
Ephemeral Session Processing	Process claim data in-memory with automatic purge after a time-to-live window or explicit user deletion.	Purged session data becomes inaccessible after expiry and no PHI persists on disk.
Metadata-Only Audit Logging	Store only metadata such as timestamps, rule counts, model versions, and session identifiers in a lightweight audit log.	Audit queries return correct metadata and sanitization middleware blocks PHI patterns.
Local-First Inference	Run AI models locally through an inference stack such as vLLM or Ollama, without external API calls for PHI-bearing data.	System remains functional offline and achieves end-to-end inference within 60 seconds for a five-page claim on a GPU-enabled setup.
Reproducible MLOps	Track experiments with MLflow and package the system with Docker for reproducible deployment.	Every model-training run is logged and the full system can be reproduced through containerized deployment.

5.2 Out-of-Scope: Explicitly Excluded Capabilities

The following capabilities are explicitly excluded from the capstone scope so that the project remains focused, feasible, and regulatorily responsible. These exclusions are deliberate rather than accidental and should be treated as firm boundaries for the academic submission.

Table 5.5: Explicitly excluded capabilities

Excluded Capability	Rationale for Exclusion	Alternative Approach in This Project
Claim adjudication or approval decisions	Final approval authority rests with insurers and requires licensing, liability frameworks, and formal regulatory accountability.	The system outputs risk indicators and recommendations only, with a clear disclaimer that it is a pre-check tool and not an adjudication system.
Fraud detection or anomaly identification	Fraud analysis needs distinct datasets, specialized modeling, and governance beyond capstone scope.	The project focuses on administrative validation such as missing fields, policy limits, and internal consistency checks.

Excluded Capability	Rationale for Exclusion	Alternative Approach in This Project
Clinical or medical judgment	Validation of medical necessity requires clinical expertise and additional regulatory oversight.	The system checks documentation completeness and policy alignment, not treatment appropriateness.
Legal interpretation of policy contracts	Contract interpretation carries legal liability and requires domain-specific legal expertise.	Explanations cite policy text but do not interpret legal meaning; users are directed back to insurers for contractual disputes.
Live integration with insurer or TPA core systems	Production integrations require API contracts, security review, and operational SLAs that are beyond a student project.	ClaimGuardian remains a standalone prototype and exports results through PDF and structured presentation only.
Multi-tenant authentication or enterprise SSO	Identity management is an infrastructure concern outside the AI core being evaluated in this capstone.	The capstone remains in a single-user demonstration mode, with authentication deferred to later enterprise phases.
Bulk claim processing or batch APIs	Bulk orchestration requires job queues, asynchronous execution, and scaling work outside the MVP boundary.	The current prototype processes one claim per session, while batch processing is documented as future enhancement.
Mobile application or responsive PWA	A web application is sufficient for academic demonstration, while mobile optimization adds design and testing overhead.	The prototype is desktop-first, with mobile adaptation noted for future work.
Multi-language UI beyond English	Full localization introduces substantial product and testing complexity within the capstone timeline.	The interface remains English-only, while the architecture can later support internationalization resources.
Guarantee of claim acceptance	No pre-submission checker can guarantee an insurer's final decision because adjudication depends on additional evidence and insurer-side workflows.	The system provides risk indicators and likely issue explanations, not guarantees of settlement.

5.3 Deferred for Enterprise Readiness (Acknowledged but Not Implemented)

The following capabilities are recognized as important for production deployment, but they are explicitly deferred in order to keep the capstone bounded. Their inclusion here demonstrates architectural foresight and provides a practical roadmap for future work.

Table 5.6: Capabilities deferred for enterprise readiness

Deferred Capability	**Why It Matters for Enterprise**	**How Architecture Anticipates It**
Policy Registry and Multi-Policy Rule Management	Enterprise deployments must manage many policy variants and corresponding rule sets.	Rules are stored as structured configuration or Python modules, so a registry becomes a data-model extension rather than a rewrite.
Role-Based Access Control and SSO	Multi-user enterprise environments require secure identity and authorization boundaries.	The FastAPI session model can later be extended with user and role attributes without changing the core validation pipeline.
Tenant Isolation for Multi-Company Deployment	Different insurers or TPAs require strict isolation of data, rules, and audit trails.	Session-level isolation already exists; a tenant identifier can be added as a logical partition key.
Bulk Processing and Async Job Queue	Operations teams often need to validate many claims simultaneously.	The pipeline already works per session, so queue-based orchestration can be layered on top of the current architecture.
Export Hooks such as Webhooks, CSV, or EDI	Enterprise systems need structured downstream ingestion of validation outcomes.	Validation outputs are already structured as JSON, making export formatters a presentation-layer extension.
Rule Versioning and Historical Audit	Policy rules evolve over time, and prior validations must remain explainable.	Rule files can be versioned and the audit log can store the rule-version hash with minimal schema changes.
SLA Monitoring and Performance Dashboards	Production systems require uptime visibility and performance monitoring.	Health checks and monitoring middleware can be added without altering the core AI pipeline.
Confidence Thresholding and Human Review Queue	Enterprise teams need routing logic for low-confidence cases.	Confidence scores are already part of the extraction output, so threshold-based routing is a lightweight extension.
Configurable Data-Retention Policies	Enterprises may need anonymized retention for analytics or compliance.	The anonymization module already separates PHI handling from storage decisions, enabling configurable retention later.

Deferred Capability	Why It Matters for Enterprise	How Architecture Anticipates It
Disaster Recovery and Backup Procedures	Production systems require backup, recovery, and operational continuity planning.	The audit store and model registry can later be migrated to enterprise-grade infrastructure without redesigning the application logic.

5.4 Scope Boundaries: Regulatory and Ethical Guardrails

Beyond functional scope, the project is constrained by privacy, transparency, and accountability requirements relevant to the Indian health-insurance setting. These guardrails operationalize the regulatory and ethical principles already identified in the problem framing and literature survey [7, 14, 1, 10, 9].

Table 5.7: Regulatory and ethical scope boundaries

Boundary	Regulatory or Ethical Basis	Implementation in Scope
No persistent PHI storage	DPDP Act 2023 data minimization and privacy-by-design expectations.	Ephemeral session handling is used so that intermediate claim data is purged after expiry or deletion.
Plain-language explanations	IRDAI expectation that claim-related reasons be communicated with clear policy grounding.	RAG-grounded explanations cite policy clauses and use deterministic prompting for stable output.
Human-in-the-loop for high-risk decisions	Responsible AI and human-oversight principles in high-stakes workflows.	The interface requires user review before final report generation and highlights high-risk findings for manual attention.
Transparency about AI limitations	Consumer protection and academic integrity considerations.	The interface explicitly states that the system is an AI-assisted pre-check tool and displays confidence scores for extracted fields.
Bias testing across meaningful subgroups	Fairness expectations in automated decision support for regulated domains.	The evaluation suite includes stratified testing by hospital type, geography, and claim category wherever data permits.
Adversarial testing for PHI leakage	Cyber-security and privacy assurance expectations for health-insurance systems.	Automated tests inject dummy PHI and verify 0% leakage in logs, outputs, and explanation text.

5.5 Feasibility Alignment: 8-Month Student Team Constraints

The scope is deliberately calibrated for a student team working part-time over roughly eight months. The intention is to deliver a technically rigorous, AI-focused capstone without overpromising on enterprise-scale implementation [4, 3].

Table 5.8: Feasibility alignment with capstone constraints

Constraint	**How Scope Addresses It**
Limited GPU and compute resources	The system prioritizes local-first inference, lightweight fallback models, and modular evaluation so that useful progress remains possible even on modest hardware.
Limited proprietary training data	Synthetic augmentation, few-shot prompting, and selective use of partner data for evaluation reduce dependence on large private datasets.
Variation in team AI and software expertise	The architecture is decomposed into independent modules with clear interfaces, allowing team members to own different components in parallel.
Academic evaluation timeline	The project is organized into demo-able increments so that each module produces a visible artifact before full integration.
Regulatory compliance complexity	Privacy-by-design choices such as ephemeral sessions, PHI masking, and metadata-only logging reduce the burden of retrofitting compliance later.
Industry partner expectations	Explicit scope boundaries prevent overpromising while the deferred-enterprise subsection still shows roadmap awareness and architectural maturity.

These scope boundaries ensure that ClaimGuardian remains feasible as a capstone project, ethically responsible in a sensitive domain, and technically credible enough to serve as a foundation for later enterprise refinement.

6 Proposed System

The proposed system, **ClaimGuardian**, is an AI-augmented, privacy-first pre-submission validation platform designed specifically for the Indian health-insurance ecosystem. It operates as a transparent decision-support layer that ingests heterogeneous claim documents, extracts structured fields through multimodal AI, validates policy-aligned conditions through deterministic and semantic reasoning, and returns confidence-weighted risk indicators together with plain-language, policy-cited explanations. The architecture is deliberately engineered for academic feasibility within an eight-month student timeline while still preserving modularity, auditability, and regulatory alignment [1, 3, 4].

6.1 System Overview and Architectural Vision

ClaimGuardian follows a hybrid AI reasoning paradigm that combines the flexibility of modern machine learning with the auditability of deterministic software. Instead of relying on a single opaque end-to-end model, the system composes specialized components into a transparent workflow that can be independently tested, monitored, and improved [1, 6, 10, 9].

Table 6.1: Architectural layers of the proposed ClaimGuardian system

Layer	**Function**	**Architectural Choice**
Document Intelligence	Layout-aware extraction from noisy, multilingual Indian medical forms	Vision-language models such as Qwen2.5-VL supported by OCR and NER fallback cascade.
Privacy Filter	PHI detection and anonymization before downstream processing	Microsoft Presidio with custom Indian PII recognizers and adversarial safety testing.
Validation Engine	Policy-rule application and document-completeness checks	LLM-assisted rule mining transformed into an executable and auditable Python rule engine.
Risk Prediction	Rejection probability and deduction estimation	Gradient Boosting style tabular ML with SHAP feature attribution for interpretability.
Explanation Generator	Plain-language, policy-grounded user feedback	Retrieval-Augmented Generation using semantic search plus constrained LLM generation.
Orchestration and MLOps	Pipeline sequencing, experiment tracking, and reproducibility	FastAPI async orchestration, MLflow tracking, Docker Compose, and ephemeral session handling.

The system explicitly does **not** perform claim adjudication, fraud detection, or clinical judgment. It functions strictly as a pre-submission readiness assistant so that all AI outputs remain confidence-scored, human-verifiable, and traceable either to source documents or to policy clauses [1, 14].

6.2 End-to-End Operational Workflow

The user-facing workflow is designed for clarity, user control, and incremental validation:

1. **Document Ingestion:** The user uploads the policy schedule, hospital bill, and discharge summary in PDF form.
2. **AI Classification and Extraction:** The system auto-classifies documents and extracts structured fields such as dates, billed amounts, diagnosis-related details, and room-rent information, along with per-field confidence scores.
3. **Privacy Sanitization:** Extracted text is passed through a PHI-detection layer, and identifiers are replaced with typed placeholders before logging or downstream reasoning.
4. **Interactive Review:** The interface highlights low-confidence fields so the user can review and correct them before validation proceeds.
5. **Rule Validation and Risk Scoring:** A deterministic rule engine checks policy-linked constraints, while a tabular ML model estimates rejection probability and likely deduction ranges.
6. **Grounded Explanation Generation:** A retrieval layer fetches relevant policy clauses, after which a constrained LLM produces plain-language explanations with explicit grounding.
7. **Report Export and Session Purge:** The user downloads a structured validation report, after which intermediate data is purged either on explicit deletion or after session expiry.

6.3 AI-Native Pipeline Architecture

The core intelligence pipeline follows a staged, fallback-resilient design in which each component operates independently but passes schema-validated outputs to the next stage. This keeps the system testable, debuggable, and resilient to document-quality variation [1, 6].

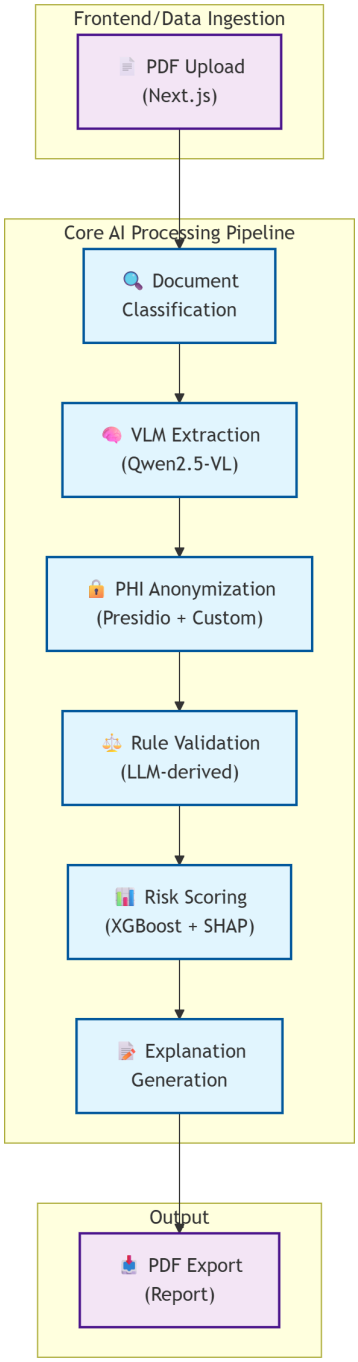

Fig. 6.1: Simplified AI-native pipeline flow for ClaimGuardian

Key Architectural Properties

- **Graceful degradation:** If the primary vision-language extraction fails or falls below the confidence threshold, the pipeline can cascade to NER, OCR, and then manual entry.
- **Contract-driven modules:** Each stage declares strict Pydantic-style input and output schemas, enabling independent implementation and testing.

- **Local-first inference:** PHI-bearing workflows avoid external API calls; inference remains on-premise through local model serving.
- **MLOps readiness:** Experiment tracking, dataset versioning, and automated benchmarking are treated as first-class engineering concerns rather than afterthoughts.

6.4 Privacy, Security and Compliance by Design

Privacy is designed into the architecture rather than retrofitted after development. This is especially important in a workflow that processes health documents, policy records, and identity-linked information [7, 10, 1].

Table 6.2: Privacy, security, and compliance controls in the proposed system

Privacy Control	Implementation	Compliance Alignment
Zero Persistent PHI	Ephemeral sessions with time-limited retention and automatic purge on completion or expiry.	DPDP Act 2023 data minimization and deletion-oriented privacy handling.
Pre-Inference Anonymization	Presidio plus custom recognizers run before LLM reasoning or log generation.	Sensitive-data handling expectations in regulated insurance workflows.
Metadata-Only Audit Trail	SQLite records timestamps, model versions, and rule counts without extracted values or PHI.	Purpose limitation, traceability, and auditability without storing sensitive content.
LLM Safety Testing	Adversarial prompt and leakage tests verify that generated outputs do not expose PHI.	Safe deployment expectations for AI systems in high-stakes domains.
No External Data Transfer	All inference is local, containerized, and transport-protected where networking is involved.	Privacy-preserving deployment aligned with consent-based and local processing principles.

6.5 Technical Stack and Deployment Strategy

The technical stack is selected to balance AI capability, zero-budget feasibility, and accessibility for a student team. The goal is to remain ambitious without depending on paid cloud APIs or proprietary infrastructure [4, 3, 1].

Table 6.3: Technical stack proposed for ClaimGuardian

Layer	Technology	Rationale
Frontend	Next.js 15, TypeScript, Tailwind CSS, shadcn/ui	Modern component-driven web interface with strong developer productivity and clean review workflows.
Backend	FastAPI with Python 3.12+ and Pydantic v2	Async-ready API design, automatic documentation, and strict schema validation.
Document AI	Qwen2.5-VL, DistilBERT NER, EasyOCR	Multimodal extraction with CPU and GPU fallback options across document-quality conditions.
Privacy	Microsoft Presidio, custom spaCy NER, regex recognizers	Open-source and extensible privacy layer for Indian PII patterns.
Reasoning and RAG	Local LLaMA or Mistral class models, BGE-M3 embeddings, FAISS	Grounded explanations without cloud dependency.
Risk Prediction	XGBoost with SHAP	Interpretable and auditable tabular ML for risk-related scoring.
Infrastructure	Docker Compose, Redis for ephemeral sessions, SQLite for metadata, MLflow	Reproducible local deployment with lightweight operational footprint.
Deployment	Fly.io, Render, or university-hosted infrastructure	Zero-cost or low-cost deployment paths with optional GPU acceleration.

Zero-Budget Execution Strategy

- All core models are open-weight and can run locally.
- No paid cloud APIs are required for PHI-bearing document processing.
- Free-tier or institute-managed hosting is sufficient for demonstration deployment.
- Synthetic data augmentation reduces dependence on large proprietary datasets.
- AI coding assistants can accelerate boilerplate implementation, while human effort remains focused on architecture, evaluation, and domain alignment.

6.6 Key Differentiators and Alignment with Academic and Industry Goals

Table 6.4: Key differentiators of the proposed ClaimGuardian system

Dimension	How ClaimGuardian Delivers
Academic Rigor	The architecture is modular and independently testable, with explicit evaluation metrics such as F1-score, ROC-AUC, grounding-oriented explanation quality, and MLOps tracking.
Industry Relevance	The system targets a real operational gap by reducing avoidable intake rework and improving submission readiness before formal claim review.
AI-Centric Design	The pipeline is natively multimodal and retrieval-grounded, rather than a simple OCR-plus-regex workflow with AI added later.
Regulatory Readiness	The design aligns with privacy-by-design, metadata-only audit practices, plain-language explanation goals, and explicit scope boundaries.
Capstone Feasibility	The project supports incremental delivery, clear completion criteria, optional stretch goals, and operation on consumer or university hardware.

6.6.1 Scope Reinforcement for Academic Integrity

The proposed system is explicitly bounded to **pre-submission validation only**. It:

- provides confidence-weighted extraction, rule validation, risk indicators, and policy-cited explanations,
- operates as a decision-support assistant with human review at every critical step,
- does not approve or reject claims, detect fraud, make clinical judgments, or integrate with live insurer systems, and
- does not guarantee claim approval, since final decisions remain with licensed insurers and TPAs.

These boundaries ensure that the project remains technically rigorous, ethically responsible, and academically defensible while still offering tangible value to policyholders and insurance-operations teams [4, 3, 1].

7 Methodology (Approach to Solve the Problem)

The project follows a **research-grounded, incremental, and privacy-by-design methodology** organized into seven sequential yet parallelizable phases. Each phase is intended to solve a specific dimension of the health-insurance claim-validation problem while maintaining academic rigor, capstone feasibility, and architectural foresight for later enterprise extension [4, 3, 1, 2].

7.1 Domain-Driven Problem Framing and Scope Calibration

Objective: Translate real-world insurance bottlenecks into precise technical requirements while enforcing clear academic and regulatory boundaries.

Approach: Structured requirement elicitation is carried out with the external domain partner to map frequent claim-friction patterns such as waiting-period violations, room-rent cap breaches, and missing documentation into formal validation logic. Out-of-scope areas such as adjudication, fraud detection, and clinical judgment are explicitly documented so that scope creep is prevented from the beginning. The expected behavior of the system is also aligned with IRDAI expectations around clear communication and with DPDP-oriented privacy principles such as data minimization and purpose limitation [3, 4, 7, 14].

Validation: This phase is considered complete once the team finalizes an approved software requirements specification, a traceability matrix linking user pain points to functional modules, explicit scope-boundary sign-off, and a compliance-mapping note for the relevant regulatory obligations.

7.2 Privacy-by-Design Data Curation and PHI-Safe Processing

Objective: Establish a secure and compliant data pipeline that can process sensitive health documents without persistent PHI storage or unintended external leakage.

Approach: Microsoft Presidio is used together with custom Indian PII recognizers for identifiers such as Aadhaar, PAN, IFSC, and Indian phone formats so that text is anonymized before downstream inference. Documents are processed through ephemeral session handling with automatic expiry and explicit user-triggered deletion. Because proprietary claim samples are limited, synthetic claim data is generated through domain rules and privacy-safe tooling to support training and evaluation without depending entirely on real patient documents [10, 1, 2, 7].

Validation: Success is measured by zero PHI leakage in logs or generated outputs under adversarial injection tests, proper documentation of the synthetic dataset strategy, and an internal privacy audit aligned with the intended DPDP-compliant workflow.

7.3 Hybrid AI Pipeline Construction (Vision to Rules to Reasoning)

Objective: Build a multi-stage, fallback-resilient AI pipeline that balances multimodal extraction with deterministic, auditable validation.

Approach: A vision-language model such as Qwen2.5-VL is used as the primary extractor for layout-aware parsing across noisy and multilingual Indian medical forms. A confidence-driven fallback chain is then defined: vision-language extraction to NER to OCR to manual entry. In parallel, an LLM-assisted rule-mining component derives policy constraints with source grounding, and a retrieval-augmented reasoning layer reduces hallucination during explanation generation. High-confidence checks are ultimately executed by a deterministic Python rule engine so that repeatability and auditability are preserved [6, 1, 9, 2].

Validation: Component-level benchmarks are used here, including at least token-level extraction accuracy, rule-mining precision, fallback activation traces, and grounding-oriented explanation scoring against source policy clauses.

7.4 Explainable Output Generation and Human-in-the-Loop Validation

Objective: Convert AI outputs into transparent, actionable guidance while preserving user oversight at critical decision points.

Approach: The interface presents extracted values together with confidence-weighted indicators so that low-confidence items can be corrected before validation. Plain-language explanations are generated through constrained prompting and grounded retrieval rather than unconstrained free-form generation. Risk scores are computed through interpretable tabular ML, and feature-attribution techniques are used to show the main drivers of a high-risk outcome. Human confirmation is required before final report generation so that the system remains a decision-support tool rather than an autonomous decision-maker [1, 9, 2].

Validation: This phase is evaluated through user-correction success rate, readability scoring for generated explanations, and consistency between model-attribution output and domain-expert review.

7.5 MLOps-Driven Experimentation and Reproducible Benchmarking

Objective: Ensure that model development remains tracked, evaluated, and reproducible across iterative improvements in a manner consistent with both academic and industry engineering standards.

Approach: Experiment tracking is maintained for hyperparameters, metrics, and model lineage, while dataset and configuration versioning are used to preserve reproducibility. Automated benchmark suites compare baseline and improved models using task-appropriate metrics such as extraction accuracy, explanation similarity, and ROC-AUC for risk scoring. Model-registry conventions are used to distinguish staging-quality models from deployment-ready checkpoints, and ablation-oriented comparisons are retained for academic reporting [2, 3].

Validation: Evidence for this phase includes a complete experiment history, reproducible environment configuration, automated benchmark reports, and clearly documented ablation studies showing why one model choice is preferred over another.

7.6 Incremental Integration, Adversarial Testing and Privacy Hardening

Objective: Assemble all modules into a cohesive pipeline while stress-testing the system against realistic failure modes, hostile inputs, and limited compute conditions.

Approach: Integration follows a modular and mutually exclusive workflow in which each subsystem exposes contract-driven schemas and can therefore be composed without hidden coupling. Adversarial tests are executed by injecting malformed PDFs, simulated OCR noise, ambiguous prompts, and PHI-like strings into the pipeline. Fallback activation, log sanitization, and expiry-based data purging are verified systematically. End-to-end tests are also run under both GPU-assisted and constrained CPU conditions to ensure that the design remains feasible for a student team with variable hardware access [3, 4, 1, 2].

Validation: This phase is validated through high integration-test pass rates, full fallback-path coverage, zero PHI in audit logs, and end-to-end latency targets that remain acceptable for both GPU and CPU execution modes.

7.7 Deployment, Evaluation and Knowledge Transfer

Objective: Deliver a reproducible and academically defensible prototype together with complete documentation, evaluation artefacts, and a clear enterprise-extension roadmap.

Approach: The final system is packaged in Docker Compose with separated frontend, backend, and model-serving responsibilities. Deployment targets can include free-tier hosting or institute-managed infrastructure depending on resource availability. The team then consolidates evaluation reports, architecture diagrams, privacy-audit outputs, and experiment-tracking artefacts into a capstone defense portfolio. Limitations, failure modes, and deferred enterprise features such as RBAC, multi-tenancy, and batch APIs are documented explicitly so that architectural maturity is demonstrated without overclaiming [4, 3, 1].

Validation: Completion of this phase is demonstrated by a reproducible deployment workflow, presentation-ready evaluation material, complete technical documentation, and a prototype that can be defended both as an engineering system and as an academic research artefact.

8 Analysis

This section presents a comprehensive analysis of the ClaimGuardian project, covering the process model adopted for development, a multi-dimensional feasibility study, a zero-budget cost analysis aligned with student constraints, and a phased timeline chart mapped to the modular architecture defined in the project plan.

8.1 Process Model Used for the Project

8.1.1 Rationale for Process Model Selection

Given the project's characteristics—**AI-native pipeline development**, **privacy-by-design constraints**, **an 8-month student timeline**, and **an enterprise-aware architecture**—a pure waterfall or pure agile approach is insufficient. Instead, ClaimGuardian adopts a **Hybrid Incremental-Agile Process Model with MLOps Integration**, combining sprint-based delivery, experiment tracking, privacy checkpoints, fallback validation, and human review into a single academically rigorous and practically demonstrable workflow.

Table 8.1: Process model adaptations for ClaimGuardian

Methodology Element	**Adaptation for ClaimGuardian**	**Justification**
Agile Sprints (2-week cycles)	Each sprint delivers a testable module, such as Module 3: VLM Extraction, with clear "done" criteria.	Enables incremental validation, early demo artifacts, and parallel team work.
MLOps Pipeline Integration	Every AI module includes experiment tracking through MLflow, model versioning, and automated evaluation gates.	Ensures reproducibility, model quality control, and academic rigor.
Privacy Gates	PHI anonymization through Presidio and log sanitization are mandatory checkpoints before any module integration.	Embeds DPDP Act 2023 compliance by design rather than as a retrofitted add-on.
Fallback Testing	Each AI component must demonstrate graceful degradation from VLM → NER → OCR → manual review.	Ensures system resilience under noisy real-world document conditions.
Human-in-the-Loop Review	Critical outputs such as extracted fields, risk flags, and explanations require explicit user confirmation.	Maintains accountability and aligns with IRDAI transparency expectations.

8.1.2 Process Workflow Diagram

SPRINT 0: FOUNDATION

- Repository scaffold, Docker setup, CI/CD baseline, and session-manager initialization.
- Privacy middleware skeleton and MLflow initialization.

↓

SPRINTS 1–6: CORE AI PIPELINE

- Sprints 1–2: Document ingestion, OCR, and PHI anonymization.
- Sprints 3–4: VLM extraction and confidence calibration.
- Sprints 5–6: Rule mining and deterministic validation.
- Each sprint ends with unit tests, an integration demo, and model-evaluation checks.

↓

SPRINTS 7–10: REASONING AND UX

- Sprints 7–8: RAG pipeline and explanation generation.
- Sprint 9: Risk classifier and SHAP-based explainability.
- Sprint 10: Review UI, results dashboard, and PDF export.
- Each sprint includes user testing and grounding validation.

↓

SPRINTS 11–12: HARDENING AND DEPLOYMENT

- Sprint 11: Privacy audit and adversarial testing.
- Sprint 12: Containerization, deployment, and documentation.
- Final demo packaging and capstone-defense preparation conclude the cycle.

Fig. 8.1: Process workflow diagram for the ClaimGuardian delivery plan

8.1.3 Key Process Artifacts

Table 8.2: Key process artifacts for delivery, privacy, and evaluation

Artifact	Purpose	Tool / Format
Sprint Backlog	Tracks module-level tasks with AI-specific acceptance criteria and sprint completion boundaries.	GitHub Projects + Markdown
ML Experiment Log	Records model training runs, hyperparameters, evaluation metrics, and lineage information.	MLflow + DVC
Privacy Audit Checklist	Verifies PHI handling at each integration point before release or demonstration.	Custom checklist + automated tests
Fallback Activation Log	Documents when and why fallback chains trigger during evaluation and integration runs.	Structured JSON logs
User Testing Protocol	Validates UI clarity, explanation readability, and correction workflow quality.	Likert-scale surveys + think-aloud sessions

8.2 Feasibility Study

A five-dimensional feasibility analysis ensures that the project remains viable within academic, technical, operational, financial, and regulatory constraints.

8.2.1 Technical Feasibility

Table 8.3: Technical feasibility assessment

Factor	Assessment	Mitigation Strategy
Model Availability	Open-weight VLMs such as Qwen2.5-VL, LLMs such as Llama-3 and Phi-4, and OCR engines such as EasyOCR are freely available.	Use Hugging Face Hub and Ollama for local inference and avoid paid external APIs.
Compute Requirements	VLM inference typically requires a GPU with at least 8 GB VRAM, while CPU-compatible models such as DistilBERT remain necessary for fallback operation.	Document minimum hardware expectations, test CPU fallback early, and leverage university GPU laboratories whenever available.
Data Availability	Proprietary samples from Mpathy are limited, and public Indian medical-document datasets remain scarce.	Augment with synthetic data using Faker and domain rules, and rely on few-shot prompting to reduce training-data dependence.

Factor	Assessment	Mitigation Strategy
Integration Complexity	The OCR → NER → VLM → RAG → rule-engine pipeline requires careful orchestration across multiple stages.	Use modular Pydantic contracts, async FastAPI orchestration, and explicit fallback chains between modules.
Privacy Engineering	PHI detection and anonymization must run before any downstream processing occurs.	Place Presidio and custom NER in middleware and validate them using an adversarial testing suite.

Conclusion: The project is technically feasible with careful scoping, explicit fallback design, and early validation of GPU and CPU execution paths.

8.2.2 Operational Feasibility

Table 8.4: Operational feasibility assessment

Factor	Assessment	Mitigation Strategy
Team Skills	The student team may have uneven levels of AI/ML expertise across modules.	Use modular ownership, pair programming, and AI coding assistants for boilerplate and rapid iteration.
Domain Knowledge	Health-insurance claim workflows involve policy language, intake rules, and documentation patterns that are not trivial to model.	Maintain close collaboration with Mpathy and document common claim-error patterns early.
Maintenance Burden	AI modules require monitoring, retraining awareness, and drift detection even in a capstone context.	Use MLflow tracking, define a simple retraining trigger, and clearly document the "as-is" capstone scope.
User Adoption	Policyholders or reviewers may distrust AI-generated pre-checks if the interface feels opaque.	Present transparent confidence indicators and explicit "pre-check only" disclaimers in the user interface.

Conclusion: The project is operationally feasible with clear role allocation, continued domain support from the partner organization, and a user-centered interaction design.

8.2.3 Economic Feasibility (Zero-Budget Strategy)

Table 8.5: Zero-budget economic feasibility assessment

Resource	Cost-Saving Approach	Estimated Savings
Cloud APIs	Local inference through Ollama or vLLM avoids external LLM calls for PHI-bearing data.	INR 0 versus INR 50,000+ for equivalent cloud API usage
GPU Compute	University laboratory GPUs, free-tier hosting for demos, and CPU fallback reduce paid infrastructure needs.	INR 0 versus INR 20,000+ for cloud GPU rental
Software Licenses	The stack is fully open-source, including FastAPI, Next.js, Presidio, MLflow, and Docker.	INR 0 versus INR 10,000+ for commercial tooling
Data Acquisition	Synthetic data generation and limited partner samples avoid the need for paid proprietary datasets.	INR 0 versus INR 30,000+ for commercial data acquisition
Deployment	Docker Compose and free-tier hosting are sufficient for the capstone demo environment.	INR 0 versus INR 15,000+ for production-grade hosting

Total Estimated Cost: INR 0–5,000 for optional items such as domain registration, printing, or minor contingency needs.

Conclusion: The project is economically feasible within student budget constraints and does not require mandatory commercial expenditure.

8.2.4 Schedule Feasibility

Table 8.6: Schedule feasibility assessment

Phase	Duration	Critical Path Items	Buffer Strategy
Foundation (Modules 0–2)	4 weeks	Docker setup, session management, and PDF upload.	Parallel frontend and backend work for non-blocking setup tasks.
Core AI Pipeline (Modules 3–7)	10 weeks	VLM extraction, PHI anonymization, and rule-engine integration.	Run fallback testing early and prepare synthetic data in parallel.
Reasoning & UX (Modules 8–11)	8 weeks	RAG explanations, risk classifier, review UI, and export workflow.	Keep template-based explanations ready as a fallback path.
Hardening & Deployment (Modules 12–14)	4 weeks	Privacy audit, containerization, and final documentation.	Defer stretch goals if the capstone timeline begins to slip.

Conclusion: The schedule remains feasible because phases intentionally overlap at integration boundaries, keeping the overall capstone delivery within a 26-week academic window.

8.2.5 Legal and Regulatory Feasibility

Table 8.7: Legal and regulatory feasibility assessment

Requirement	Compliance Approach	Verification Method
DPDP Act 2023 (Data Minimization)	Use ephemeral sessions and ensure PHI is anonymized before storage or logging.	Run adversarial PHI-injection tests and audit log-sanitization behaviour.
IRDAI Guidelines (Plain-Language Explanations)	Generate RAG-grounded explanations with policy-linked citations and human-readable wording.	Review explanation clarity through sampled expert checks and human evaluation.
No Adjudication Claim	Keep explicit UI disclaimers and ensure outputs are advisory risk indicators rather than claim decisions.	Validate scope wording in the interface and supporting documentation.
Academic Data Use	Restrict partner-supplied materials to testing and avoid persistent storage of sensitive records.	Maintain documented data-usage boundaries and use synthetic data for training-oriented workflows.

Conclusion: The project is legally and regulatorily feasible because compliance is treated as an architectural constraint rather than a late-stage documentation exercise.

8.3 Cost Analysis

8.3.1 Resource Breakdown (Zero-Budget Execution)

Table 8.8: Zero-budget resource breakdown

Resource Category	Item	Quantity	Unit Cost	Total Cost	Notes
Compute	Local GPU (university lab)	1 machine × 8 months	INR 0	INR 0	Fallback to CPU execution if a GPU is unavailable.
Cloud Hosting	Fly.io or Render free tier	1 instance × 3 months	INR 0	INR 0	Sufficient for a controlled capstone demo.
Software	Open-source stack	All components	INR 0	INR 0	Includes FastAPI, Next.js, Presidio, MLflow, Docker, and related tooling.
Data	Synthetic data + Mpathy samples	500 synthetic + 50 real documents	INR 0	INR 0	No paid datasets are required.
Development Tools	GitHub, VS Code, Ollama, Docker	Standard student toolchain	INR 0	INR 0	Uses free or student-accessible tooling only.

Resource Category	Item	Quantity	Unit Cost	Total Cost	Notes
Documentation	LaTeX, Markdown, draw.io, Mermaid	Open-source tools	INR 0	INR 0	Documentation remains fully reproducible and version-controlled.
Contingency	Domain registration, printing, minor expenses	Optional	INR 5,000 max	INR 0–5,000	Not required for academic submission.

Grand Total: INR 0–5,000, with all mandatory project components executable at zero direct cost.

8.3.2 Human Effort Estimation (Student Team)

Table 8.9: Human effort estimation for the student team

Role	Team Size	Hours / Week	Total Hours (8 months)	Key Responsibilities
AI / ML Engineer	2	15	~960	VLM extraction, rule mining, risk classifier development, and RAG pipeline work.
Backend Developer	1	12	~384	FastAPI orchestration, session management, and privacy middleware implementation.
Frontend Developer	1	12	~384	Next.js interface, review form, results dashboard, and PDF export workflow.
Domain / Testing Lead	1	8	~256	Mpathy coordination, test-data curation, user testing, and compliance checks.
Project Coordinator	1 (shared)	4	~128	Sprint planning, documentation control, and demo preparation.

Total Person-Hours: ~2,112 hours (approximately 264 person-days).

Feasibility Note: This workload remains achievable through part-time student commitment averaging about 50 team-hours per week.

8.3.3 Cost-Benefit Summary

Table 8.10: Cost-benefit summary

Benefit Category	Quantifiable Impact	Qualitative Impact
Academic Value	One capstone project with scope for 4–6 publications, posters, or presentations.	Deepens understanding of AI pipeline design, MLOps, and privacy engineering.

Benefit Category	Quantifiable Impact	Qualitative Impact
Industry Relevance	Prototype can support internal testing by Mpathy and strengthen internship or collaboration pathways.	Demonstrates an enterprise-aware architecture within realistic academic constraints.
Skill Development	Provides hands-on experience with VLMs, RAG, Presidio, FastAPI, Next.js, and deployment tooling.	Produces a portfolio-ready system relevant to AI/ML and full-stack roles.
Social Impact	Can reduce avoidable claim-submission friction for Indian policyholders if extended beyond the capstone stage.	Aligns with consumer-protection and transparency goals in the insurance ecosystem.

ROI Assessment: The project offers high academic and learning return on investment, negligible financial risk, and a credible pathway toward industry relevance.

8.4 Timeline Chart

8.4.1 High-Level Gantt Chart (26-Week Capstone Timeline)

Phase / Week	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
Phase 1: Foundation & Core Pipeline (Modules 0–7)	•	•	•	•	•	•	•	•	•	•	•	•	•													
Phase 2: Reasoning & UX (Modules 8–11)												•	•	•	•	•	•	•	•							
Phase 3: Hardening & Deployment (Modules 12–14)																				•	•	•				
Buffer: Documentation, Demo, and Submission Prep																							•	•	•	•

Fig. 8.2: High-level Gantt chart for the 26-week ClaimGuardian capstone timeline

8.4.2 Module-Level Milestone Map

Table 8.11: Module-level milestone map

Module	Target Week	Deliverable	Success Metric
0: Scaffold	Week 2	`docker compose up` works and the `/health` endpoint responds.	CI passes and all core services start reliably on a clean machine.
1: Session Manager	Week 3	Session CRUD API with a 30-minute TTL.	Sessions remain isolated and auto-purge works correctly.
2: PDF Upload	Week 4	Upload endpoint stores files in a session-scoped temporary directory.	Valid PDFs are accepted and invalid uploads are rejected gracefully.
3: VLM Extraction	Week 7	Qwen2.5-VL extracts structured fields with confidence scores.	At least 85% token-level F1 on internal test documents.
4: PHI Anonymization	Week 8	Presidio plus custom NER masks all sensitive identifiers before downstream processing.	Zero PHI leakage in logs or generated outputs on the test suite.
5: Rule Engine	Week 10	Deterministic rules validate waiting periods, room-rent constraints, and document completeness.	Every rule is traceable back to policy text or an explicit project assumption.
6: Explanation Generator	Week 12	Template-based explanations are generated for rule-based flags.	Human readability score of at least 4.0 / 5.0 in internal review.
7: Review UI	Week 13	Editable review form with confidence indicators for extracted fields.	Users correct at least 90% of low-confidence fields during testing.
8: Risk Classifier	Week 15	XGBoost predicts rejection probability.	At least 80% ROC-AUC on the evaluation split.
9: RAG Explanations	Week 17	Explanations cite retrieved policy clauses through grounded generation.	At least 80% grounding quality under BERTScore-oriented review.
10: PDF Report	Week 18	Downloadable report consolidates validation findings and recommendations.	Report remains PHI-safe and understandable to a non-technical user.
11: Pipeline Orchestrator	Week 19	End-to-end workflow completes from upload to extraction, validation, and explanation.	The full pipeline completes in under 60 seconds on GPU-assisted runs.
12: Audit Logger	Week 20	Metadata-only SQLite audit log is available for traceability.	Zero PHI appears in any log entry.

Module	Target Week	Deliverable	Success Metric
13: Privacy Hardening	Week 21	Log sanitization and session auto-purge are verified under adversarial testing.	Privacy tests pass and TTL-based purging is consistently enforced.
14: Containerization	Week 22	`docker compose up` deploys the full system on a fresh machine.	One-command deployment works without external proprietary dependencies.
Buffer: Documentation & Demo	Weeks 23–26	Capstone report, presentation, final documentation, and demo preparation.	The project becomes submission-ready and defense-ready.

8.4.3 Critical Path and Risk Mitigation

Table 8.12: Critical path risks and mitigation strategy

Critical Path Item	Risk	Mitigation
VLM extraction accuracy on Indian documents	Low accuracy on noisy scans or heterogeneous hospital-document formats.	Start testing early on Mpathy samples and keep OCR plus NER fallback paths ready.
GPU availability for inference	University GPU access may be limited or unpredictable.	Validate CPU-compatible fallback models early and document performance expectations clearly.
PHI anonymization completeness	Presidio may miss Indian PII patterns such as Aadhaar or PAN variants.	Add custom recognizers and adversarial test cases for domain-specific identifiers.
RAG explanation grounding	LLM output may hallucinate policy citations or oversimplify complex coverage rules.	Use deterministic prompting, schema-constrained output, and sampled human review.
Timeline slippage	Interdependent modules may delay downstream integration milestones.	Keep modules MECE, defer stretch goals, and run weekly scope reviews.

9 System Design

This section presents the architectural design of ClaimGuardian through formal modeling artifacts: a multi-level Data Flow Diagram (DFD), key UML diagrams (Use Case, Class, and Sequence), and an Enhanced Entity-Relationship (EER) diagram. These artifacts collectively document the system's structural composition, behavioral workflows, and data relationships while preserving the privacy-first, AI-native, and modular design principles established in earlier sections.

9.1 Data Flow Diagram (DFD)

9.1.1 Context Diagram (Level 0)

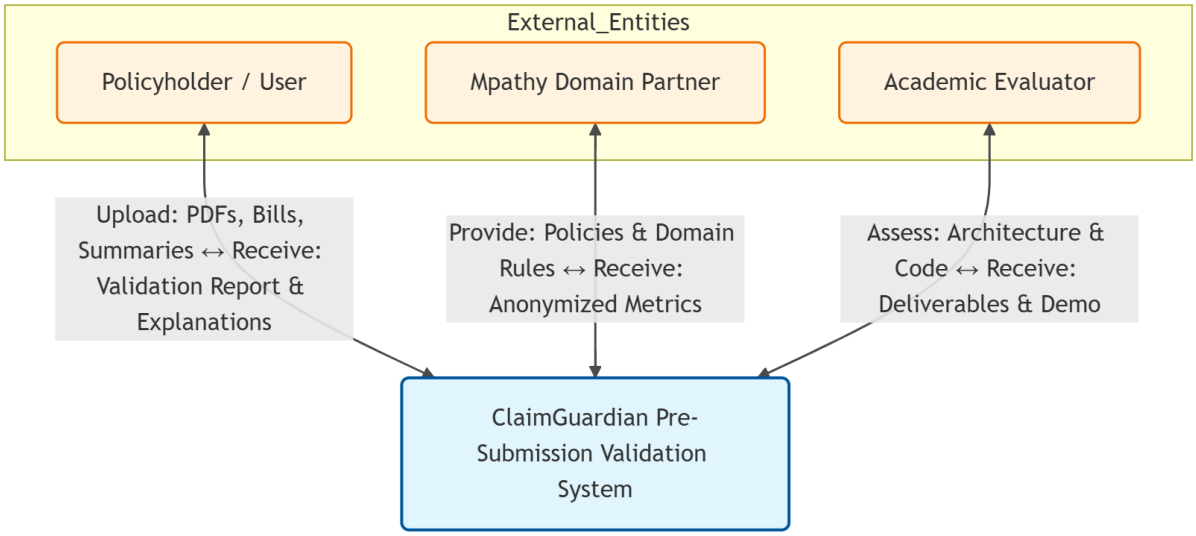

Fig. 9.1: Context diagram for ClaimGuardian (Level 0 DFD)

Table 9.1: External entities and data flows in the context diagram

Entity	Input to System	Output from System	Purpose
Policyholder	Claim documents in PDF form and user corrections.	Validation report, risk score, and policy-cited explanations.	Primary user receiving pre-submission guidance before insurer submission.
Mpathy Partner	Proprietary sample policies, domain-rule feedback, and workflow insight.	Aggregated anonymized metrics with no PHI and model-performance summaries.	Supports domain validation and iterative improvement of the system.

Entity	Input to System	Output from System	Purpose
Academic Evaluator	Assessment criteria and rubric requirements.	Source code, architecture documents, demo output, and evaluation artefacts.	Evaluates the capstone implementation and its academic rigor.

9.1.2 Level 1 DFD: Core Processing Pipeline

The Level 1 DFD is presented on the following page in landscape orientation for readability.

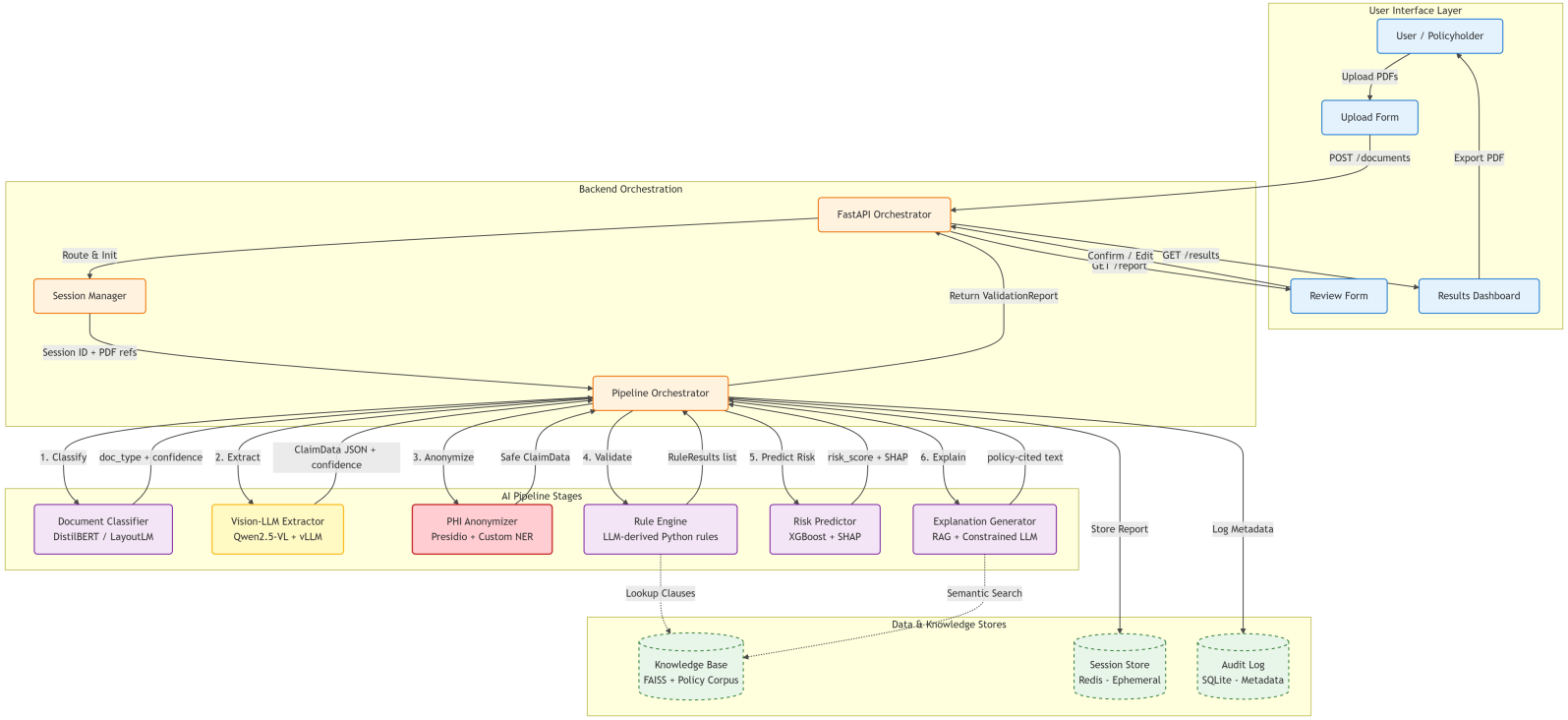

Fig. 9.2: Core processing pipeline (Level 1 DFD)

9.1.3 Level 2 DFD: Privacy-Preserving Data Handling

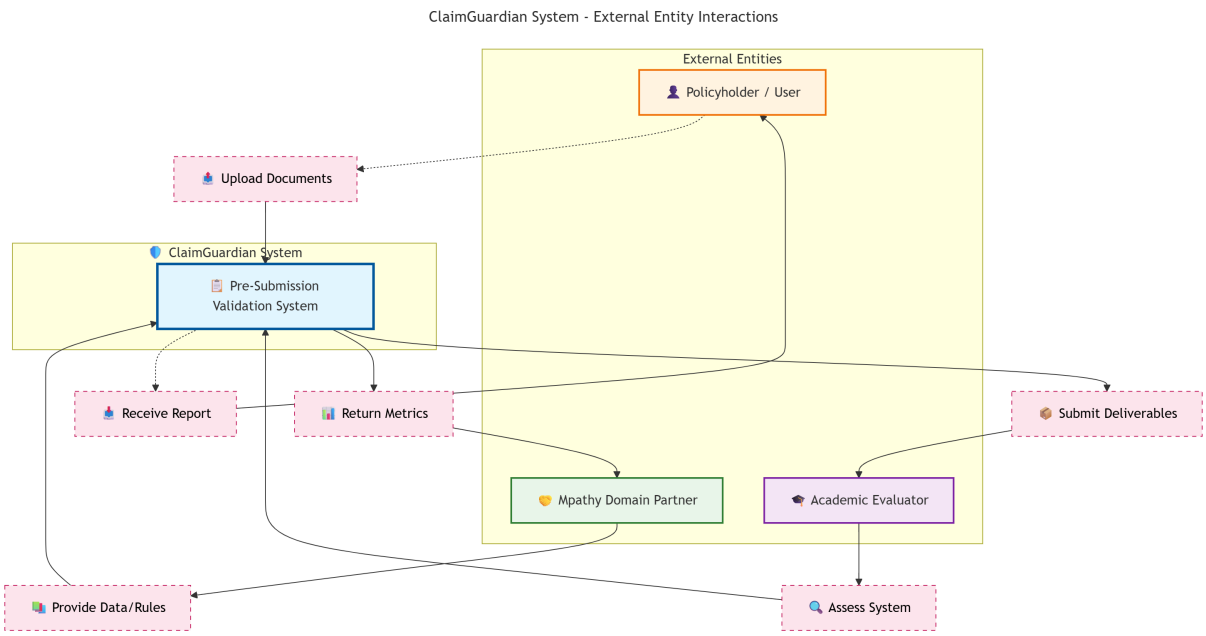

Fig. 9.3: Privacy-preserving data handling (Level 2 DFD)

Key Privacy Design Decisions

1. **Pre-inference anonymization:** PHI detection runs before any LLM reasoning or logging, preventing leakage at the source.
2. **Typed token replacement:** Tokens such as `[PERSON_001]` preserve syntactic structure for downstream NLP while preventing direct re-identification.
3. **Ephemeral session store:** Redis-backed in-memory storage with automatic TTL ensures no persistent PHI is retained, while manual user deletion triggers immediate purge.
4. **Metadata-only audit:** Audit logs contain session identifiers, timestamps, and aggregate metrics rather than raw extracted values, rule messages, or PHI-bearing text.

9.2 UML Diagrams

9.2.1 Use Case Diagram

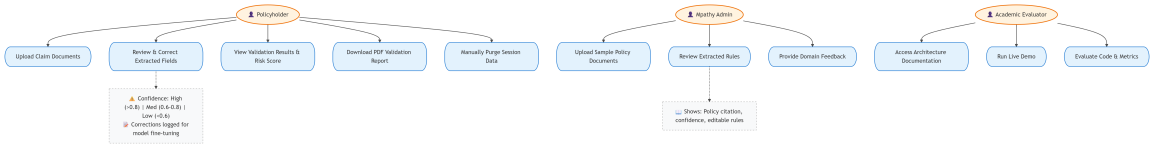

Fig. 9.4: Use case view of ClaimGuardian actors and interactions

9.2.2 Class Diagram (Core Domain Model)

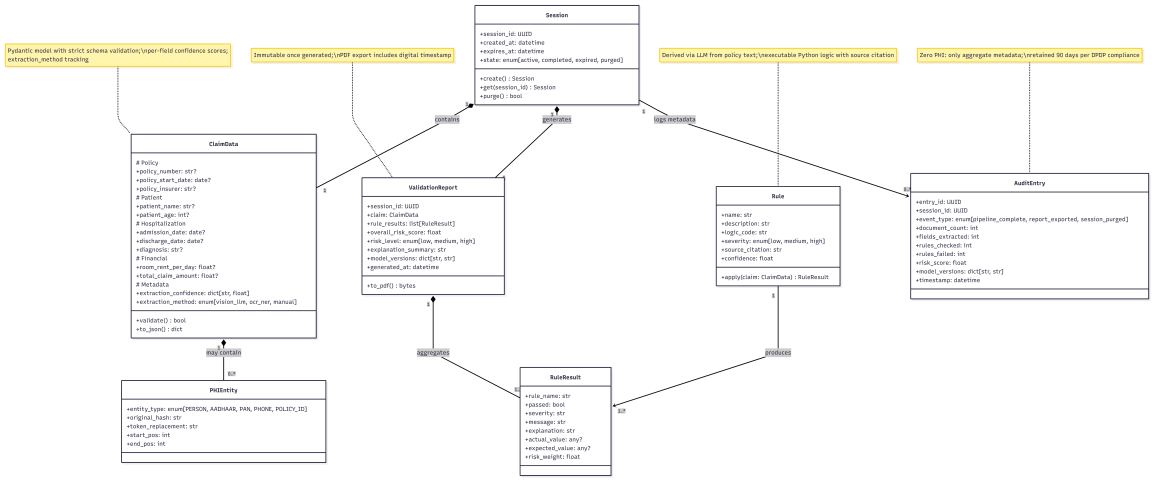

Fig. 9.5: Core domain model for sessions, claims, rules, and reports

9.2.3 Sequence Diagram: End-to-End Validation Workflow

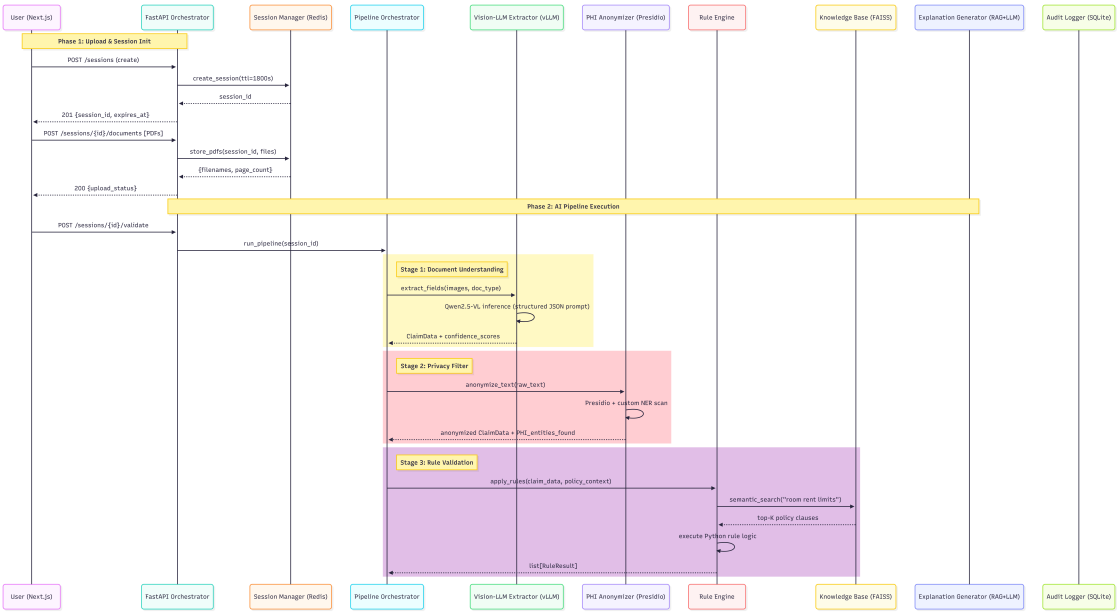

Fig. 9.6: End-to-end sequence for upload, validation, explanation, and export

9.3 Enhanced Entity-Relationship (EER) Diagram

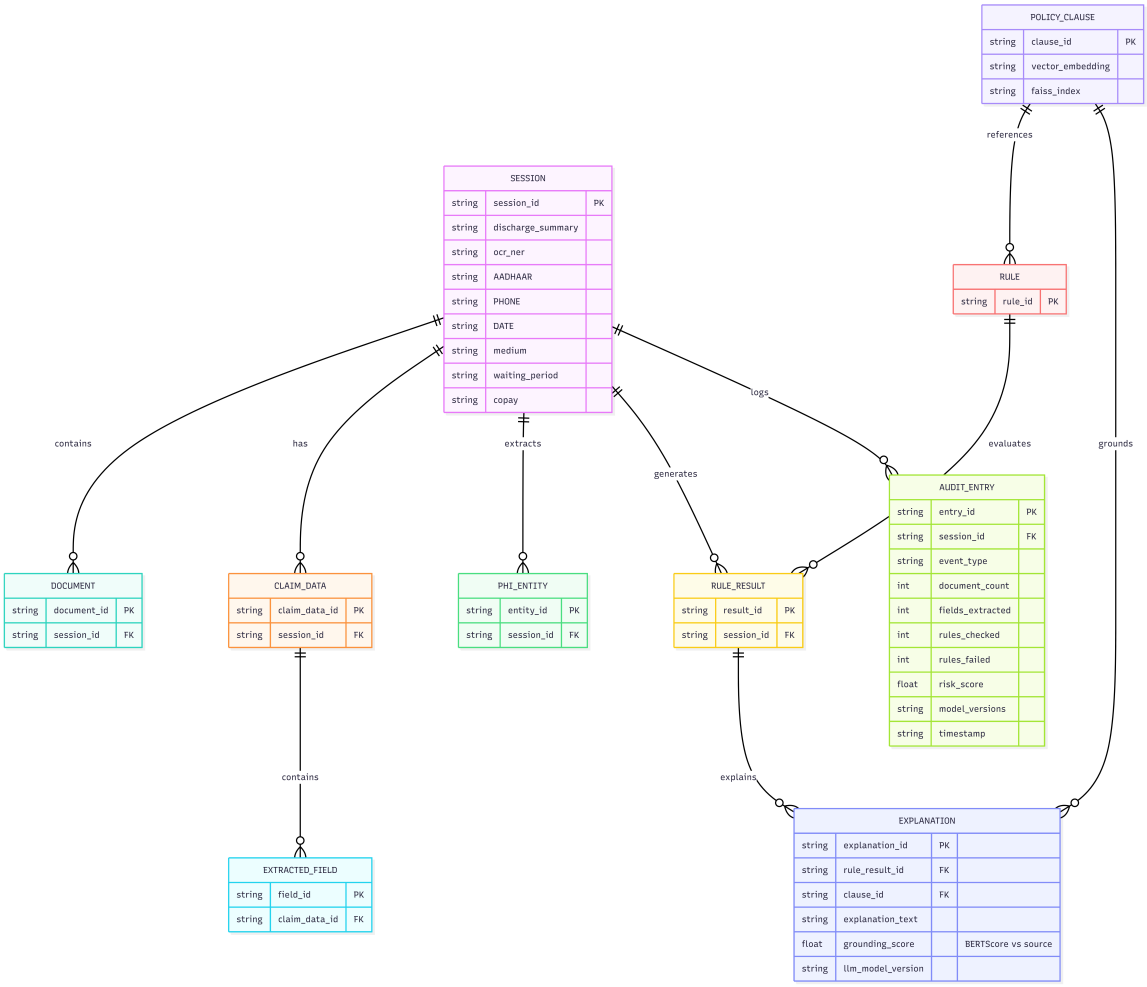

Fig. 9.7: Enhanced entity-relationship model for ClaimGuardian

9.3.1 EER Design Rationale

Table 9.2: EER design rationale

Design Choice	**Justification**
Disjoint specialization for `DOCUMENT`	Each uploaded file is treated as exactly one document type, such as bill, discharge summary, or policy, preventing ambiguous downstream classification.
Typed `PHI_ENTITY` specialization	Enables targeted anonymization rules for Aadhaar, PAN, phone numbers, policy identifiers, and related categories while improving compliance reporting.
Irreversible hash for PHI values	Supports DPDP-aligned traceability without allowing reconstruction of the original sensitive value from audit data.

Design Choice	Justification
Embedding vector in `POLICY_CLAUSE`	Enables efficient semantic retrieval through FAISS for grounding rule execution and explanation generation.
Grounding score in `EXPLANATION`	Makes explanation quality measurable and supports filtering of low-confidence or weakly grounded outputs.
Metadata-only `AUDIT_ENTRY`	Separates operational logging from sensitive data storage, simplifying privacy audits and compliance checks.

9.3.2 Database Implementation Notes

- **Primary storage:** Redis is used for ephemeral session data, while SQLite stores metadata-only audit records.
- **Vector index:** FAISS is used as a local file-based store for policy clause embeddings, avoiding mandatory external database dependencies.
- **Schema enforcement:** Strict schema validation is handled through Pydantic models at the application layer, while persistence remains intentionally minimal.
- **Migration strategy:** The audit log can be migrated from SQLite to PostgreSQL for enterprise deployment without requiring application-logic changes.
- **Backup policy:** Only the audit log and policy corpus require backup; session data is ephemeral by design and therefore excluded from durable backup routines.

9.4 Design Validation Against Requirements

Table 9.3: Design validation against requirements

Requirement ID	Design Artifact Addressing It	Verification Method
FR-2 (VLM Extraction)	Sequence diagram: VE stage; class diagram: extraction-method tracking in ClaimData.	Unit-test that VLM output validates against the strict ClaimData schema.
FR-6 (PHI Safety)	Level 2 DFD: PHI detection flow; EER: hashed PHI-entity representation.	Adversarial test: inject PHI and verify 0% leakage in outputs and logs.
FR-7 (RAG Explanations)	Sequence diagram: EG stage; EER: explanation grounding-score attribute.	Evaluate generated explanations with a BERTScore threshold of at least 0.8.
NFR-2 (No PHI Persistence)	Level 2 DFD: session lifecycle; EER: TTL-based session constraints.	Integration test: force session purge and verify Redis keys and temporary files are deleted.

Requirement ID	Design Artifact Addressing It	Verification Method
NFR-5 (MLOps)	Class diagram: model-version tracking in ValidationReport and AuditEntry.	Verify that MLflow or equivalent model-version identifiers are logged for reproducibility.
Scope: Pre-Submission Only	Use case diagram shows no approve-or-reject insurer action path.	Code and scope review confirm that no adjudication logic exists in the rule-engine module.

10 Hardware and Software Requirements (Technology Stack)

This section specifies the technical infrastructure required to develop, test, and deploy Claim-Guardian. Requirements are calibrated to balance **AI capability**, **zero-budget feasibility**, **student-team accessibility**, and **enterprise architectural readiness**. All components are open-source or free-tier compatible, with clear upgrade paths for production deployment.

10.1 Hardware Requirements

10.1.1 Development Environment (Student Team)

Table 10.1: Development environment requirements for the student team

Component	Minimum Specification	Recommended Specification	Rationale
CPU	4-core `x86_64` processor such as Intel i5 or AMD Ryzen 5.	8-core processor such as Intel i7 or AMD Ryzen 7.	Enables parallel model inference together with frontend and backend development.
RAM	16 GB DDR4.	32 GB DDR4.	Required for loading 7B–13B parameter models more comfortably and reducing out-of-memory failures during experimentation.
GPU	None for CPU-only fallback development.	NVIDIA RTX 3060 / 4060 with 12 GB VRAM or better.	Enables local VLM or LLM inference through vLLM, while CPU fallback relies on DistilBERT and EasyOCR.
Storage	256 GB NVMe SSD.	512 GB NVMe SSD.	Fast I/O improves model loading, PDF processing, and Docker-layer caching.
Network	Broadband connection of at least ≥ 25 Mbps.	Fiber connection of at least ≥ 100 Mbps.	Supports downloading large model weights and synchronizing artefacts with Hugging Face or similar repositories.

Zero-Budget Strategy

- Leverage university computer laboratories that provide shared GPU workstations.
- Use Google Colab Free Tier with T4-class GPU access for occasional fine-tuning tasks.
- Maintain CPU-only compatibility for every module except high-throughput VLM inference, which has a fallback path.

10.1.2 Testing & Evaluation Environment

Table 10.2: Testing and evaluation environment

Component	Specification	Purpose
Test Server	Same as the recommended development environment.	Supports end-to-end pipeline testing under realistic latency conditions.
Evaluation Dataset Storage	50 GB of available space.	Stores synthetic claims, Mpathy sample files, and gold-label annotations for benchmarking.
Adversarial Testing Rig	Any development-class machine.	Injects PHI patterns, malformed PDFs, and prompt-attack style inputs to verify safety and robustness.

10.1.3 Deployment Environment (Capstone Demo)

Table 10.3: Deployment environment for the capstone demo

Component	Free-Tier Option	Paid Upgrade Path	Notes
Hosting Platform	Fly.io shared CPU instances or Render free-tier deployment.	AWS EC2 `t3.medium` or GCP `e2-medium`.	Free tiers are sufficient for a demo with roughly $\leq$ 5 concurrent users.
GPU Inference	CPU-only fallback through DistilBERT and EasyOCR.	RunPod or Lambda Labs with pricing in the range of USD 0.20–0.40 / hr for high-end GPU rental.	VLM inference benefits from a GPU and should be documented as an optional stretch path for the demo.
Vector Database	FAISS local file-based index.	Pinecone free tier or Weaviate Cloud.	FAISS is fully local and avoids mandatory external services.
Session Store	Redis in-memory storage with ephemeral session state.	Redis Cloud free tier or managed Redis service.	Ephemeral by design; no persistence is required for capstone operation.
Audit Database	SQLite file-based metadata store.	PostgreSQL through a Fly.io or similar managed add-on.	Metadata-only persistence makes SQLite sufficient at capstone scale.

Deployment Constraint: All PHI must be processed in memory and purged after session expiry; no persistent storage of sensitive data is permitted on any deployment tier.

10.2 Software Requirements

10.2.1 AI / ML Stack (Core Intelligence Layer)

Table 10.4: AI / ML stack for the core intelligence layer

Component	Technology	Version	Purpose	Zero-Budget Alternative
Vision-Language Model (Primary Extraction)	Qwen2.5-VL-7B via Ollama or vLLM.	Project-pinned model release.	Layout-aware field extraction from scanned Indian medical documents.	Fall back to EasyOCR plus DistilBERT NER on CPU-compatible hardware.

Component	Technology	Version	Purpose	Zero-Budget Alternative
Document Classification	DistilBERT-base-uncased (fine-tuned).	Transformers 4.40+	Auto-classifies uploaded documents such as bills, policies, and discharge summaries.	Use the pre-trained checkpoint and fine-tune only on synthetic data if compute is limited.
Named Entity Recognition (PHI Detection)	spaCy 3.7+ with Microsoft Presidio 2.2+ and custom Indian PII recognizers.	spaCy 3.7+, Presidio 2.2+	Detects and anonymizes Aadhaar, PAN, phone numbers, and patient names.	Presidio remains fully open-source and can be extended with regex-based Indian formats.
Domain Reasoning LLM	LLaMA-3-8B-Instruct or Phi-4 through Ollama or vLLM.	Project-pinned model release.	Supports policy-rule mining, explanation generation, and risk reasoning.	Use smaller 7B-class models with GGUF quantization for CPU-oriented inference.
Embeddings & Semantic Search	BGE-M3 or `all-mpnet-base-v2`	Sentence Transformers 2.5+	Converts policy clauses and claim context into vectors for RAG retrieval.	Fully local embedding generation avoids any API cost.
Vector Index	FAISS.	1.8+	Stores and retrieves policy clauses for grounded semantic search.	File-based FAISS indexing requires no external vector database.
Risk Prediction Classifier	XGBoost or scikit-learn tabular classifiers.	XGBoost 2.0+ or scikit-learn 1.4+	Predicts claim-rejection probability from extracted structured features.	Pure CPU execution keeps the tabular ML layer lightweight and affordable.
Explainability (SHAP)	SHAP.	0.44+	Attributes risk predictions to input features for transparency.	Open-source integration with XGBoost and scikit-learn keeps explainability low cost.
Model Fine-Tuning	PEFT (LoRA) with Hugging Face TRL and Accelerate.	Accelerate 0.30+	Enables efficient domain adaptation of LLMs for insurance-claim language.	LoRA reduces VRAM requirements substantially and can run on 12 GB GPUs.
Experiment Tracking	MLflow.	2.12+	Tracks training runs, metrics, model versions, and lineage.	Self-hosted MLflow avoids any cloud subscription cost.

AI Stack Justification

- All models are open-weight and runnable locally through Ollama or vLLM, which removes API costs and supports DPDP-aligned local processing.
- Fallback chains preserve functionality even without GPU access: VLM → DistilBERT NER → EasyOCR → manual entry.
- Quantization support through formats such as GGUF and AWQ helps 7B-class models run on consumer hardware with limited accuracy loss.

10.2.2 Backend & API Layer

Table 10.5: Backend and API layer requirements

Component	Technology	Version	Purpose	Rationale
API Framework	FastAPI.	Python 3.12+	Provides an async REST API with automatic OpenAPI documentation.	High performance, straightforward testing, and deep Pydantic integration.
Data Validation	Pydantic v2.	Python	Enforces schemas for ClaimData, RuleResult, and ValidationReport.	Supports type safety and automatic JSON-schema generation.
Async Task Queue	Celery with Redis.	Celery 5.3+	Offloads long-running inference tasks such as LLM reasoning.	Prevents API timeouts and enables progress-aware asynchronous execution.
Session Management	Redis 7.2+.	In-memory key-value store	Stores session state, extracted fields, and intermediate results.	Auto-expiry through TTL enforces privacy-by-design behaviour.
Audit Logging	SQLite with SQLAlchemy.	SQLite 3.44+, SQLAlchemy 2.0+	Persists metadata-only audit entries for reproducibility.	Lightweight file-based storage avoids the need for an external database.
PDF Processing	PyMuPDF with Pillow.	PyMuPDF 1.24+, Pillow 10.2+	Converts PDF pages into images for VLM and OCR ingestion.	Supports high-fidelity rendering and robust document pre-processing.
Inference Orchestration	Python `async/await` with Pydantic-based contracts.	Custom project logic.	Coordinates OCR, NER, LLM, RAG, and rule-engine stages in one pipeline.	Makes module boundaries explicit, testable, and easier to evolve incrementally.

10.2.3 Frontend & User Interface

Table 10.6: Frontend and user-interface requirements

Component	Technology	Version	Purpose	Rationale
Framework	Next.js.	15 with React 18.3+ and TypeScript 5.4+	Provides the main web application and server-rendered frontend.	Modern developer experience with routing, SSR, and TypeScript support.
UI Components	`shadcn/ui` with Tailwind CSS.	Tailwind CSS 3.4+	Supplies accessible, customizable UI primitives and styling.	Zero-cost, community-maintained, and TypeScript-friendly.
Form Handling	React Hook Form with Zod.	React Hook Form 7.51+, Zod 3.23+	Enables real-time field validation with AI confidence indicators.	Strong TypeScript integration keeps form schemas aligned with backend contracts.
Real-Time Updates	Socket.IO.	4.7+	Streams inference progress and long-running task updates to the UI.	Improves user experience during AI-heavy processing.
Visualization	Recharts with D3.js.	Recharts 2.12+, D3.js 7.8+	Renders risk gauges, confidence heatmaps, and coverage charts.	Lightweight, composable, and compatible with SSR workflows.

Component	Technology	Version	Purpose	Rationale
State Management	TanStack Query.	5.28+	Synchronizes API-backed state, caching, retry, and loading behaviour.	Reduces client-state complexity and keeps server-driven data flows predictable.

10.2.4 Infrastructure & DevOps

Table 10.7: Infrastructure and DevOps requirements

Component	Technology	Version	Purpose	Zero-Budget Strategy
Containerization	Docker with Docker Compose.	Docker 24.0+, Compose 2.24+	Creates reproducible local and demo environments.	Free tooling eliminates "works on my machine" inconsistencies.
CI / CD	GitHub Actions.	YAML workflows	Automates linting, testing, and build verification on every push.	Free tiers are sufficient for student-scale continuous integration.
Model Registry	Hugging Face Hub with MLflow Model Registry.	Current supported releases	Tracks fine-tuned checkpoints and supports rollback.	Free tiers support academic experimentation and lightweight model governance.
Data Versioning	DVC.	3.48+	Tracks datasets and model artefacts for reproducible experimentation.	Open-source integration with Git keeps data lineage low cost.
Monitoring	Prometheus with Grafana.	Prometheus 2.48+, Grafana 10.2+	Collects metrics for latency, errors, and resource utilization.	Self-hosted monitoring remains free at capstone scale.
Load Testing	Locust or `wrk`.	Locust 2.20+	Simulates concurrent users and validates latency under load.	Open-source tooling is scriptable and suitable for academic benchmarking.

10.2.5 Privacy, Security & Compliance

Table 10.8: Privacy, security, and compliance requirements

Component	Technology	Version	Purpose	Compliance Alignment
PHI Detection	Microsoft Presidio with custom spaCy NER and Indian regex recognizers.	Presidio 2.2+, spaCy 3.7+	Detects and masks Aadhaar, PAN, phone numbers, patient names, and policy identifiers before processing.	Aligns with DPDP-style data minimization and purpose-limited handling of sensitive personal data.
Log Sanitization	Custom FastAPI middleware with regex and NER redaction logic.	Project-defined implementation.	Intercepts and sanitizes application logs before sensitive data can be written.	Reduces the chance of PHI leakage through debug, audit, or exception traces.

Component	Technology	Version	Purpose	Compliance Alignment
Encryption	TLS 1.3 in transit and optional SQLCipher-based SQLite encryption.	Deployment-dependent.	Protects data in transit and optionally secures audit metadata at rest.	Supports baseline security safeguards expected under privacy-oriented regulation.
Adversarial Testing	Custom Python safety suite with prompt-injection and leakage scenarios.	Project-defined implementation.	Verifies that models do not leak PHI or hallucinate unsafe policy logic.	Supports safe deployment practice and responsible AI governance.
Session Purge	Redis TTL with a background cleanup scheduler.	Session TTL of 30 minutes.	Auto-deletes session data and enforces ephemeral processing guarantees.	Supports data minimization, purpose limitation, and deletion expectations.

10.3 Zero-Budget Execution Strategy

Table 10.9: Zero-budget execution strategy

Resource	Free / Student Option	Limitations	Mitigation
GPU Compute	University lab GPUs and Google Colab Free Tier with T4-class access.	Limited hours, shared usage, and queue wait times.	Prioritize CPU-compatible modules first and use quantized models where possible.
Cloud Hosting	Fly.io free tier and Render free-tier deployment.	Limited RAM per instance and sleep-after-idle behaviour.	Optimize Docker images and use warm-up checks before demonstrations.
Model Weights	Hugging Face Hub open-weight models.	Large downloads can require significant bandwidth and local storage.	Use model caching and document the minimum bandwidth expectations.
Vector Database	FAISS local file-based index.	No distributed scaling.	This is sufficient for a small policy corpus and has a documented upgrade path to managed vector search.
CI / CD Minutes	GitHub Actions free-tier minutes.	Heavy model training could exhaust the monthly allowance.	Run expensive model training locally and use CI primarily for code quality checks.
Domain Registration	`.vercel.app` or `.fly.dev` subdomains.	No custom branded domain by default.	Treat a custom domain as an optional stretch item for presentation polish only.

Total Estimated Cost: INR 0–2,000 for optional items such as printing, domain registration, or minor contingency.

10.4 Upgrade Path for Enterprise Deployment

Table 10.10: Upgrade path for enterprise deployment

Component	Capstone (Free)	Enterprise Upgrade	Benefit
Inference Serving	Ollama or vLLM on a single machine or single GPU.	Kubernetes-backed autoscaling with vLLM and NVIDIA Triton.	Supports high concurrency, SLA-oriented serving, and stronger operational isolation.
Vector Search	FAISS local file index.	Pinecone or Weaviate cloud with replication.	Delivers lower-latency semantic search and easier multi-region scaling at enterprise volume.
Session Store	Redis with ephemeral single-node state.	Redis Cluster with persistent audit integration to PostgreSQL.	Improves high availability and broader compliance-oriented operational durability.
Model Registry	Local or lightweight MLflow tracking.	Enterprise MLflow deployment with gated CI or Databricks-style registry controls.	Strengthens model governance, approvals, rollback safety, and lifecycle visibility.
Monitoring	Self-hosted Prometheus and Grafana.	Datadog, New Relic, or similar managed observability platforms.	Adds alerting, richer dashboards, and stronger incident-response capability.
Authentication	None for open academic demo mode.	OAuth2 / OIDC with RBAC and auditable access controls.	Enables multi-tenant separation, controlled access, and stronger regulatory readiness.

Architectural Foresight: The capstone stack is designed around clean module boundaries and contract-driven interfaces so that free-tier components can later be replaced by enterprise services without rewriting core business logic.

10.5 Compatibility & Dependency Management

Table 10.11: Compatibility and dependency management

Requirement	Specification	Enforcement
Python Version	Python 3.12+ is required for Pydantic v2 compatibility and modern async features.	Enforce through `pyproject.toml` constraints and CI environment checks.
Node.js Version	Node.js 20.x LTS is required for the planned Next.js 15 frontend environment.	Enforce through `.nvmrc` and the GitHub Actions build matrix.
Docker Base Image	`python:3.12-slim` and `node:20-alpine` are preferred for multi-stage builds.	Keep images pinned and validate them during CI builds.
Model Formats	GGUF for CPU inference, AWQ or GPTQ for GPU-optimized LLMs, and ONNX for embeddings where appropriate.	Document conversion scripts and loading expectations in `tools/` or equivalent project utilities.
Dependency Pinning	Use `poetry.lock` and `package-lock.json` or equivalent lock files.	Maintain deterministic dependency snapshots for reproducible builds.
Cross-Platform	Linux is primary, macOS is acceptable for development, and Windows is supported through WSL2.	Use Docker to standardize environments and document WSL2-specific setup constraints where needed.

References

[1] From Vision to Verdict: A Hybrid AI Framework for Compliant and Explainable Indian Health Insurance Claims Processing, 2026. Uploaded literature review.

[2] AI-Driven Health-Insurance Claim Validation System (Capstone Project Plan), 2026. Internal deep-research report.

[3] ClaimGuardian — Requirements Elicitation and Project Plan, 2026. Internal project requirements document, 10 April 2026.

[4] ClaimGuardian Project Team. A Pre-Submission Validation System for Health Insurance Claims (India), 2026. Project proposal, 23 March 2026.

[5] Guillaume Jaume, Hazim Kemal Ekenel, and Jean-Philippe Thiran. FUNSD: A Dataset for Form Understanding in Noisy Scanned Documents. *arXiv preprint arXiv:1905.13538*, 2019.

[6] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, et al. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923*, 2025.

[7] Ministry of Law and Justice, Government of India. The Digital Personal Data Protection Act, 2023, 2023. Act No. 22 of 2023, published in the Gazette of India on 11 August 2023.

[8] Rohit Saxena, Alessandro Suglia, and Pasquale Minervini. VLM-RobustBench: A Comprehensive Benchmark for Robustness of Vision-Language Models. *arXiv preprint arXiv:2603.06148*, 2026.

[9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[10] Microsoft. Presidio: Data Protection and De-identification SDK, 2026. Official documentation.

[11] Insurance Regulatory and Development Authority of India. Annual Report 2023-24, 2024. Official annual report, updated 23 December 2024.

[12] India Today Business Desk. 11% of All Health Insurance Claims Rejected, Premiums to Remain High: IRDAI Report. *India Today*, 2024. 26 December 2024.

[13] Neelanjit Das. Health Insurance Claim: 43% Policyholders Faced Difficulties, Some Had to Wait an Extra Day at Hospital, Survey. *The Economic Times*, 2024. 8 May 2024.

[14] Insurance Regulatory and Development Authority of India. FAQs on Health Insurance Regulations, 2024. Official FAQ stating that denial or repudiation communication must specifically state reasons and corresponding policy conditions.

[15] Business Today. Health Insurance: Claim Rejections by Insurance Companies Went Up by 19.10% in FY24. *Business Today*, 2024. 26 December 2024.